MALAYSIA

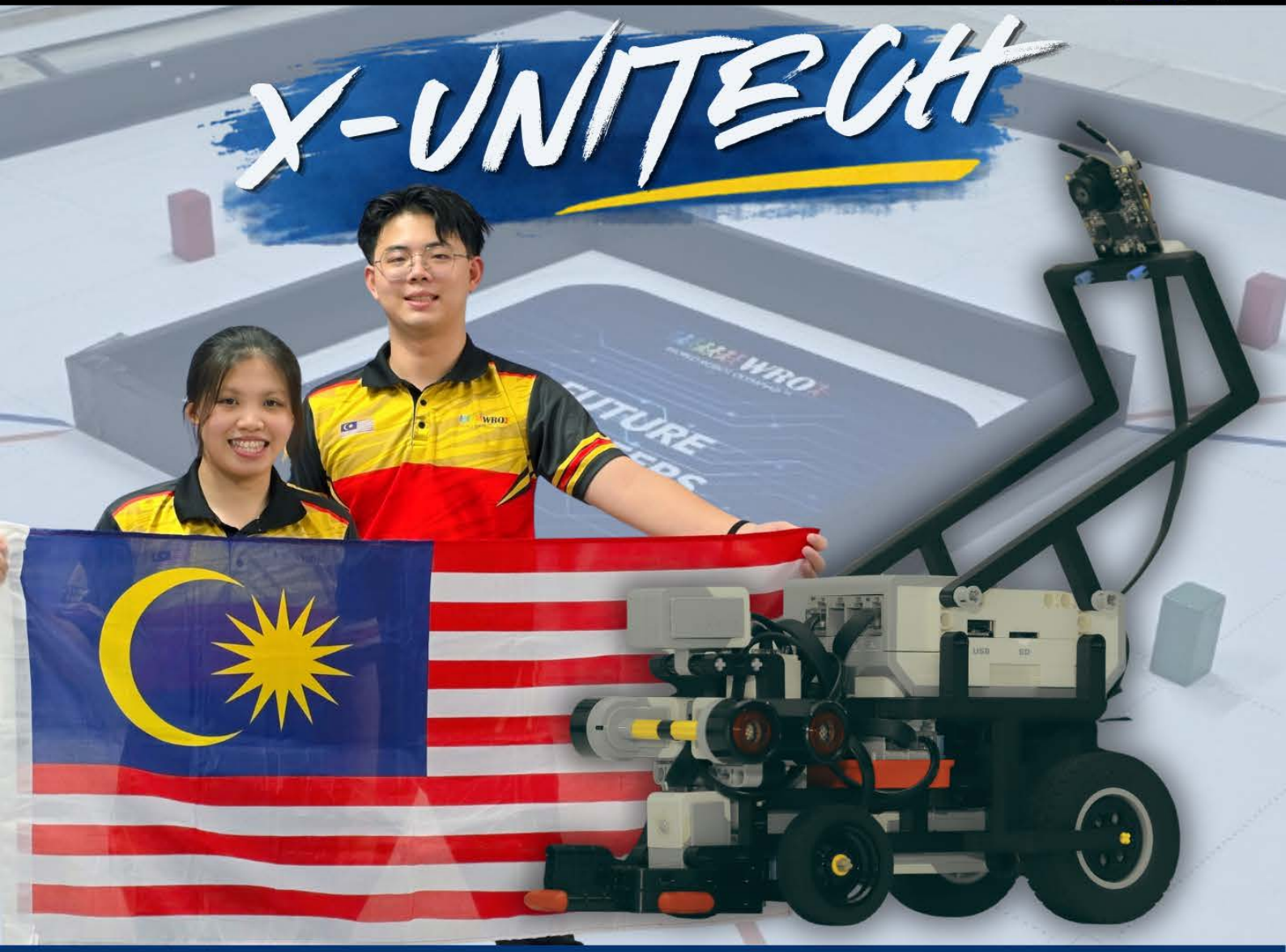
ROBOT DEVELOPMENT AND ITERATION CYCLE (GENERATION 1-4)

WORLD ROBOT OLYMPIAD ™

WRO®

PUERTO RICO 2026

ROBOTS MEET CULTURE



Team Members: RAYMOND TING YIH WEI & CARMEN HENG JIA MIN

1.0 Generation 1: Initial Robot Design and Performance Testing

1.1 Mechanical Design and Mobility System

1.1.1 Robot Configuration and Hardware Layout

1.1.1.1 Chassis Structure and Mechanical Layout

1.1.1.2 Drivetrain and Gear Transmission

1.1.1.3 Steering Mechanism and Motor Configuration

1.1.2 Motor Selection and Performance Calculations

1.1.2.1 Drive Motor Output Torque, Rotational Speed and Theoretical Velocity

1.1.2.2 Steering Motor Specifications and Direct-Drive System

1.1.3 Mobility Performance Testing

1.1.3.1 Open Challenge: Speed and Reliability

1.1.3.2 Obstacle Challenge: Navigation and Parking Exit

1.1.4 Mechanical Limitations and Design Decisions

1.2 Power Distribution and Sensor System Architecture

1.2.1 Power Supply and Hardware Connections

1.2.2 Battery Configuration and Supply Voltage

1.2.3 Sensor Selection and Functions

1.2.4 Sensor Placement and Field of View

1.2.5 Sensor Calibration

1.2.5.1 Pixy Camera Colour Recognition

1.2.5.2 Gyroscope Heading Calibration

1.2.5.3 Ultrasonic Distance Calibration

1.2.6 Sensor Edge Case

1.2.6.1 Pixy Camera Colour-Detection Accuracy

1.2.6.2 Gyroscope Heading Error and Cumulative Deviation

1.2.6.3 Ultrasonic Distance Accuracy and Detection Range

1.2.7 Sensor Limitations and Failure Analysis

1.2.8 Sensor Testing Results and Improvement Decisions

1.3 Software Design and Autonomous Navigation

1.3.1 Programming Environment and Development Platform

1.3.2 Open Challenge Software Architecture and Control Logic

1.3.2.1 Control System Architecture

1.3.2.2 Sensor Input Processing and Feedback Control

1.3.2.2.1 Camera-Based Direction Detection

1.3.2.2.2 Ultrasonic Distance Control and Steering Correction

1.3.2.2.3 Gyroscope-Based Heading Control

1.3.2.2.4 Motor Encoder-Based Steering and Distance Control

1.3.2.3 Multi-Sensor Integration and Control Decisions

1.3.2.4 Software Modules and Functional Design

1.3.2.4.1 Pixy Camera Signal Acquisition

1.3.2.4.2 Steering Control and Feedback

1.3.2.4.3 Distance and Heading Based Steering Correction

1.3.2.4.4 Encoder-Based Driving Distance Control

1.3.2.4.5 Main Navigation Sequence and Cycle Management

1.3.2.4.6 Multithreading and Concurrent Control

1.3.2.5 Open Challenge Navigation State Machine

1.3.2.6 Navigation Algorithms and Control Rationale

1.3.2.6.1 Camera Vision-Based Direction Selection Algorithm

1.3.2.6.2 Ultrasonic-Based Steering Correction Algorithm

1.3.2.6.3 Gyro-Based Heading Correction Algorithm

1.3.2.6.4 Integrated Navigation and Feedback Control Algorithm

1.3.2.7 Open Challenge Testing and Performance Results

1.3.2.7.1 Initial Direction Detection Reliability Test

1.3.2.7.2 Close-Range Wall Detection and Avoidance Test

1.3.2.8 Sensor-Related Edge Cases and Failure Analysis

1.3.3 Obstacle Challenge Software Architecture and Decision Logic

1.3.3.1 Control Architecture and Obstacle Handling

1.3.3.2 Open Challenge and Obstacle Challenge: Software Comparison

1.3.3.3 Obstacle Challenge Software Modules and Responsibilities

1.3.3.3.1 RGB Data Acquisition from the Camera

1.3.3.3.2 Camera-Based Object Detection and Position Extraction

1.3.3.3.3 Object Signature Identification and Position Tracking

1.3.3.3.4 Continuous Floor and Object Detection

1.3.3.3.5 Multi-Position Colour Sensing for Wall Detection

1.3.3.3.6 Integrated Traffic Handling and Recovery Control

1.3.3.4 Obstacle Detection, Avoidance and Recovery Algorithms

1.3.3.4.1 Ultrasonic-Based Wall Avoidance Algorithm

1.3.3.4.2 Camera-Based Relative Object Proximity Estimation

1.3.3.4.3 Colour and Position-Based Object Avoidance Algorithm

1.3.3.4.4 Motor-Speed-Based Stuck Detection and Reverse Recovery

1.3.3.5 Obstacle Challenge Navigation State Machine

1.3.3.6 Obstacle Avoidance Algorithm Testing and Performance Evaluation

1.3.3.7 Edge-Case Handling and Navigation Failure Analysis

1.4 Generation 1 Evaluation and Design Improvements

2.0 Generation 2: Design Refinement and Engineering Validation

2.1 Overview

2.2 Mechanical Redesign and Mobility Improvements

2.2.1 Chassis Configuration and Wheel Selection

2.3 Steering System Redesign and Performance Evaluation

2.3.1 Generation 1 Steering Limitations

2.3.2 Geared-Down Transmission and Ackermann Steering Geometry

2.3.3 Steering Design Concept and Engineering Rationale

2.3.4 Steering Performance Testing and Design Selection

2.3.4.1 Steering Angle Response and Accuracy

2.4 Sensor Placement and Detection Performance

2.4.1 Camera Position and Field-of-View Improvement

2.4.2 Ultrasonic Sensor Orientation Experiment

2.5 Sensor Architecture and Multi-Sensor Integration

2.5.1 Sensor Configuration and Functional Allocation

2.6 Software Redesign and Control Architecture

2.6.1 Open Challenge

2.6.1.1 Software Architecture and Control Flow

2.6.1.2 Software Redesign and Feedback Control Integration

2.6.1.3 Navigation and Steering Control Algorithms

2.6.1.3.1 Heading Error Calculation

2.6.1.3.2 Gyro-Based Heading Correction

2.6.1.3.3 Wall-Distance Correction

2.6.1.3.4 Integrated Steering Correction and Motor Feedback

2.6.1.3.5 Navigation Direction Detection and Heading Updates

2.6.1.4 Open Challenge Performance Evaluation

2.6.2 Obstacle Challenge

2.6.2.1 Software Architecture and Obstacle Navigation Control

2.6.2.2 Software Redesign and Multi-Sensor Integration

2.6.2.3 Obstacle Detection, Avoidance and Navigation Algorithms

2.6.2.3.1 Wall Detection and Avoidance Algorithm

2.6.2.3.2 Camera-Based Pillar Detection and Position Estimation

2.6.2.3.3 Position-Based Pillar Avoidance and Steering Correction

2.6.2.3.4 Parking Lot Detection and Parking Control

2.6.2.3.5 Integrated Steering Control Algorithm

2.6.2.3.6 Encoder-Based Movement Control

2.6.2.3.7 Floor-Colour Sensor-Based Navigation Direction Detection

2.6.2.4 Obstacle Challenge Performance Evaluation

2.6.2.4.1 Obstacle Navigation Testing and Results Analysis

2.7 Edge-Case Handling and Control Conflict Management

2.7.1 Multiplexer Communication Failure and Safe-Stop Response

2.7.2 Steering Correction Prioritisation and Conflict Resolution

2.8 Generation 2 Limitations and Engineering Decisions

3.0 Generation 3: Design Optimisation and Control System Development

3.1 Overview

3.2 Mechanical and Sensor Configuration Redesign

3.2.1 Front Chassis Extension for Parking Manoeuvrability

3.2.2 Steering Mechanism and Motor Configuration

3.2.3 Ultrasonic Sensor Orientation and Mounting Configuration

3.2.4 Camera Position and Field-of-View Optimisation

3.2.5 Front Steering Wheels and Tyre Selection

3.3 Software Architecture and Control System Development

3.3.1 Open Challenge Navigation Control

3.3.1.1 Sensor-Based Navigation Direction Selection

3.3.1.2 Repeated Navigation Sequence

3.3.1.3 Gyroscope-Based Heading Control

3.3.2 Programming Algorithms and Control Implementation

3.3.2.1 Sensor Initialisation and Multiplexer Configuration

3.3.2.2 Robot Position Estimation and Coordinate Tracking

3.3.2.3 Steering Reset and Reference Calibration

3.3.2.4 PID-Based Steering Control

3.3.2.5 Drive Control and Wall Following

3.3.1.6 Main Program and Navigation Sequence

3.3.3 Obstacle Challenge Software Architecture and Control Algorithms

3.3.3.1 System Initialisation and Sensor Configuration

3.3.3.2 Robot Position Estimation and Coordinate Tracking

3.3.3.3 Steering Reset and Reference Calibration

3.3.3.4 PID-Based Steering Control

3.3.3.5 Integrated Drive and Obstacle-Tracking Control

3.3.3.6 Supporting Movement Control Functions

3.3.3.7 Main Program and Obstacle Navigation Sequence

3.3.3.8 Obstacle Challenge Control Architecture Summary

3.3.4 Performance Testing and Evaluation

3.3.4.1 Open Challenge Testing and Reliability Analysis

3.3.4.2 Obstacle Challenge Testing and Performance Analysis

3.3.5 Generation 3 Limitations and Engineering Decisions

3.3.5.1 Steering Wheel Traction and Directional Stability

3.3.5.2 Gyroscope Heading Accuracy and Sensor Placement

3.3.5.3 LEGO Chassis Constraints and Structural Redesign

3.3.5.4 Camera Field of View and Detection Range

3.3.5.5 Cable Management and Internal Component Layout

3.3.5.6 Engineering Decision Summary and Implications for Generation 4

4.0 Generation 4: Design Optimisation and Control System Development

4.1 Overview

4.2 Mechanical Redesign and Mobility Enhancement

- 4.2.1 Mechanical Design Comparison Between Generation 3 and Generation 4
- 4.2.2 Wheel Selection and Traction Improvement
- 4.2.3 Integrated 3D-Printed Chassis Design
- 4.2.4 Sensor Relocation and Custom Camera Mount Design
  - 4.2.4.1 Gyroscope Relocation and Mounting Design
  - 4.2.4.2 Custom 3D-Printed Camera Mount Design
- 4.2.5 Power Budget and Power Distribution
  - 4.2.5.1 Power Budget Specification
  - 4.2.5.2 Battery Capacity Analysis
  - 4.2.5.3 Power Budget Evaluation

4.3 Software Architecture

4.4 Open Challenge Performance Evaluation

4.5 Obstacle Challenge Performance Evaluation

4.6 Final Design Overview

- 4.6.1 Mechanical Design and Mobility
- 4.6.2 Sensor Architecture and Integration
- 4.6.3 Power System and Software Architecture
- 4.6.4 Performance Evaluation
- 4.6.5 Overall Evaluation

Introduction

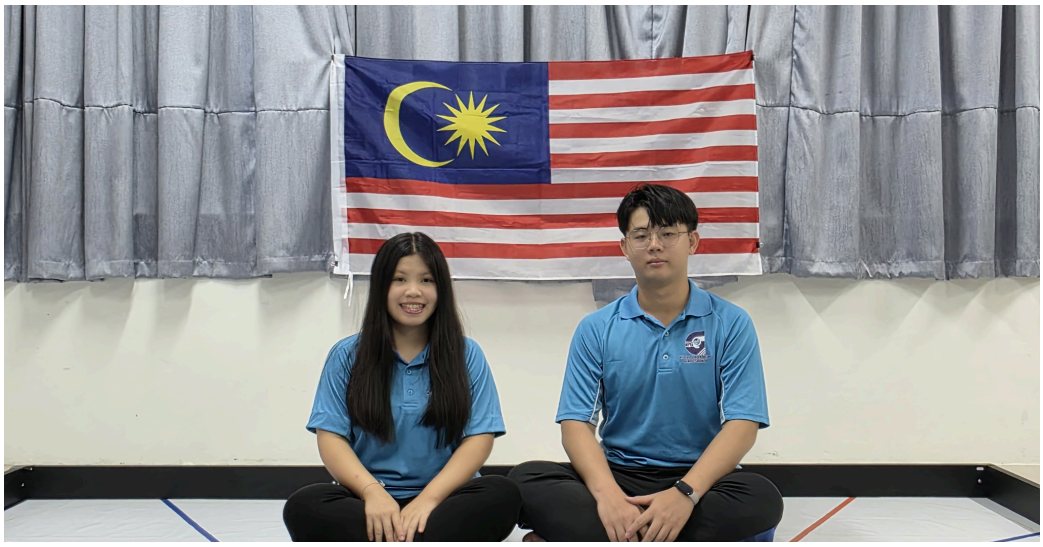
This Robot Development and Iteration Cycle documents our team's engineering process throughout the development of our robot for the World Robot Olympiad International Final in Puerto Rico 2026 in the Future Engineers category. Rather than focusing only on the final robot, it records the design decisions, engineering considerations, testing, failures, modifications and improvements made throughout the development process. Our robot was developed through an iterative engineering process. We began with an initial concept and developed several generations of prototypes. Each generation was tested to evaluate its mobility and mechanical design, power and sensor architecture, Software Architecture and Obstacle Strategy, Systems Thinking and Engineering Decisions. The results from each test were then analysed and used to identify limitations and determine the changes required for the next generation.

X-UniTech Team, Malaysia

Team Members:

- Raymond Ting Yih Wei
- Carmen Heng Jia Min

X-UniTech Team is a two-member Malaysian robotics team participating in the Future Engineers category. Our team was formed with a shared interest in robotics, engineering and technology, with the goal of designing, building and continuously improving a competitive robotic system.



1.0 Generation 1: Initial Robot Design and Performance Testing

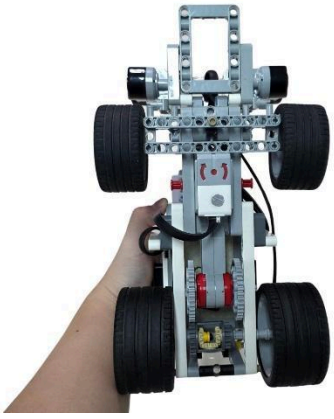
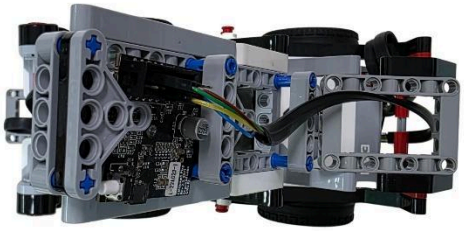
1.1 Mechanical Design and Mobility System

1.1.1 Robot Configuration and Hardware Layout

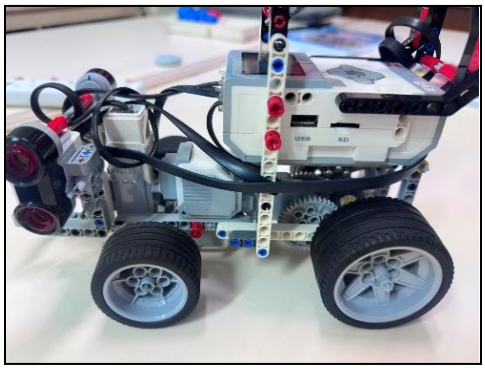
The first-generation robot was developed using the LEGO MINDSTORMS EV3 Education Set (45544) as the primary construction platform. The design was built entirely using LEGO components, allowing the robot to be assembled, modified and tested for prototyping stage.

Component	Quantity	Function
Lego Mindstorms Ev3 Education Set 45544	1 set	Main construction platform
Ev3 Large Motor	1	Driving the robot
Ev3 Medium Motor	1	Steering mechanism
Pixy Camera	1	Visual detection and object recognition
Gyro Sensor	1	Measuring orientation and rotational movement
Ultrasonic Sensor	2	Distance and obstacle detection
Lego Chassis	1	Supporting the mechanical and electronic components

The following section presents the physical design and hardware configuration of our first-generation robot:

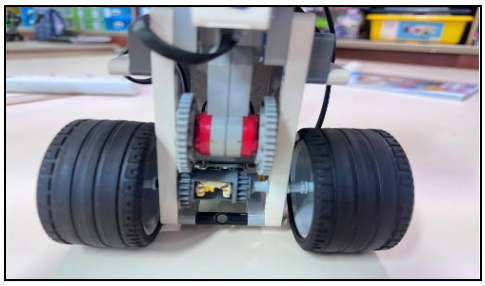
Front View	Rear View
Left View	Right View
 Bottom View	 Top View

1.1.1.1 Chassis Structure and Mechanical Layout



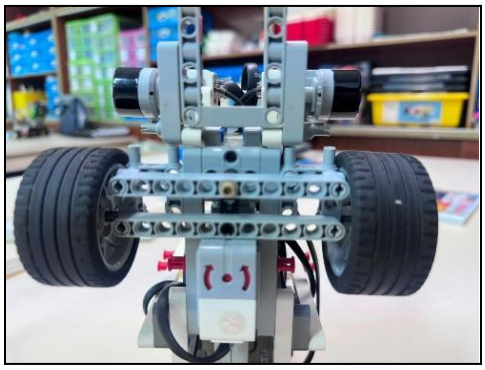
The chassis was constructed entirely using LEGO Technic components from the LEGO MINDSTORMS EV3 Education Set 45544. The structure was designed to provide a rigid platform for the drivetrain, steering mechanism, EV3 brick and sensors. The chassis was kept modular to allow components to be easily mounted, removed and repositioned during testing. The first-generation chassis measured approximately 27 cm in length, 14 cm in width and 29 cm in height, with an overall robot mass of approximately 1 kg. The main design priority was to provide sufficient structural support while keeping the construction simple and adjustable for further iterations.

1.1.1.2 Drivetrain and Gear Transmission



The driving system uses one EV3 Large Motor to power the two main driving wheels. The driving wheels are 68.8 × 36 mm, providing a relatively large contact area for traction and allowing the robot to travel a greater distance per wheel rotation. Power from the Large Motor is transmitted through a 40-tooth gear to a 24-tooth gear, followed by a differential gear. The differential allows the left and right driving wheels to rotate at different speeds during turning, helping the robot make smoother turns and reducing wheel slip. The Large Motor operates at approximately 175 rpm with a no-load current of 60 mA. The selected drivetrain was intended to provide a balance between driving speed and sufficient traction for the approximately 1 kg robot.

1.1.1.3 Steering Mechanism and Motor Configuration



The steering system uses one EV3 Medium Motor to control the direction of the steering wheels. The steering wheels measure 56 × 28 mm and are connected to the steering mechanism through a LEGO Technic linkage. The Medium Motor operates at approximately 260 rpm with a no-load current of 80 mA. Its higher rotational speed compared with the Large Motor allows the steering mechanism to respond quickly to steering commands. The steering mechanism was designed to provide directional control while keeping the mechanical structure compact. However, testing of the first-generation robot later showed that the maximum steering angle was limited, which affected the robot's manoeuvrability when avoiding obstacles.

1.1.2 Motor Selection and Performance Calculations

The motor selection was based on the different mechanical requirements of the driving and steering systems.

Motor	Function	Speed	Running Torque	Stall Torque
EV3 Large Motor	Driving	175rpm	20 N cm	40 N cm
EV3 Medium Motor	Steering	260rpm	8 N cm	12 N cm

1.1.2.1 Drive Motor Output Torque, Rotational Speed and Theoretical Velocity

The EV3 Large Motor was selected to drive the robot because the robot had an approximate mass of 1 kg. Driving requires higher torque to overcome the robot's weight, wheel resistance and drivetrain friction. The Large Motor provides approximately 20 N·cm of running torque, which is significantly higher than the Medium Motor's 8 N·cm. Therefore, the Large Motor was more suitable for propulsion.

Parameter	Value
Wheel Size	**68.8 × 36 mm**
Wheel Diameter	**68.8 mm**
Gear Combination	**40T → 24T**
Gear Ratio	**1.67:1**
Motor Speed	**175 rpm**
Theoretical Output Speed	**105 rpm**
Motor Running Torque	**20 N cm**
Theoretical Output Torque	**33.3 N cm**
Theoretical Linear Speed	**0.378 m/s**

Gear ratio:

$$\frac{40}{24} = 1.67:1$$

Output speed:

$$175 \times \frac{24}{40} = 105\ rpm$$

Output torque:

$$20 \times \frac{40}{24} = 33.3\ N{\cdot}cm$$

Wheel circumference:

$$\pi \times 68.8 = 216.1\ mm$$

Theoretical speed:

105 × 216.1 ÷ 60000 = 0.378 m/s

The 68.8 × 36 mm driving wheels were selected to provide sufficient traction and a reasonable distance travelled per rotation. The 40T-to-24T gear reduction increases the available torque while reducing wheel speed. The calculated output was approximately 105 rpm and 33.3 N·cm of theoretical torque. This provided sufficient driving performance, but the resulting theoretical

speed of 0.378 m/s was relatively high for precise obstacle navigation.

1.1.2.2 Steering Motor Specifications and Direct-Drive System

The EV3 Medium Servo Motor was selected for steering because the steering mechanism has a much lower mechanical load than the drivetrain. Instead of requiring high torque, the steering system benefits from fast and controlled rotation. With an operating speed of approximately 260 rpm, the Medium Motor can respond quickly to steering commands while being smaller and lighter than the Large Motor.

Parameter	Value
Wheel Size	**56 × 28 mm**
Wheel Diameter	**56 mm**
Motor Speed	**260 rpm**
Running Torque	**8 N·cm**
Gear Ratio	**1:1**
Output Speed	**260 rpm**
Output Torque	**8 N cm**

Calculation

Since the steering motor is directly connected:

$$Gear\ Ratio\ =\ 1{:}\,1$$

Therefore:

$$Output\ Speed\ =\ 260{\times}1\ =\ 260\ rpm$$

And:

$$Output\ Torque\ =\ 8{\times}1\ =\ 8\ N{\cdot}cm$$

Convert torque:

$$8\ N{\cdot}cm\ =\ 0.08\ N{\cdot}m$$

Steering Response

Motor speed:

$$260 \div 60 = 4.33\, rev/s$$

Therefore, the motor shaft can rotate approximately 4.33 revolutions per second. The steering system uses an EV3 Medium Motor with direct drive, without any intermediate gear reduction. Therefore, the motor speed and torque are transferred directly to the steering mechanism at a 1:1 ratio. The system provides 260 rpm and 8 N cm of running torque. The direct-drive configuration reduces mechanical complexity and backlash, allowing the steering system to respond quickly to control commands. However, testing showed that the steering angle was limited by the mechanical structure, rather than the motor speed. This reduced the robot's manoeuvrability when exiting the parking area and avoiding obstacles.

1.1.3 Mobility Performance Testing

1.1.3.1 Open Challenge: Speed and Reliability

Motor Speed (%)	Total Testing Round	Average Completion Time（s）	Successful Round	Success Rate (%)
60	10	49	10	100
80	10	25	9	90
100	10	18	2	20

The results showed that increasing the motor speed reduced the average completion time from 49 seconds at 60% speed to 18 seconds at 100% speed. However, the success rate decreased from 100% to 20%. This indicates a trade-off between driving speed and reliability. Although 100% speed provided the fastest completion time, 60% speed provided more consistent performance. Based on the testing results, 60% motor speed was considered more suitable for reliable navigation. Although 100% speed achieved a shorter completion time, its low success rate made it unsuitable for consistent performance.

1.1.3.2 Obstacle Challenge: Navigation and Parking Exit

Test	Condition	Total Rounds	Successful Rounds	Success Rate (%)
Parking Exit	Clockwise	5	1	20
Parking Exit	Counter-clockwise	5	2	40
Pillar Avoidance	Red + Green	5	1	20
Pillar Avoidance	Red + Red	5	4	80
Pillar Avoidance	Green + Green	5	4	80
Pillar Avoidance	Green Only	5	5	100
Pillar Avoidance	Red Only	5	5	100

The Obstacle Challenge was tested under different parking exit directions and pillar combinations to evaluate the robot's steering manoeuvrability and obstacle avoidance performance. The parking exit achieved success rates of 20% in the clockwise direction and 40% in the counter-clockwise direction, showing that the robot had difficulty making tight turns. For pillar avoidance, the Red + Green combination achieved only 20% success, while two same-colour pillars achieved 80% and single-pillar conditions achieved 100%. These results helped identify limitations in the robot's steering movement and camera detection during more complex obstacle arrangements.

1.1.4 Mechanical Limitations and Design Decisions

Based on the mechanical testing results, several design decisions were made for the next-generation robot.

Issue Identified	Engineering Decision
High motor speed reduced reliability	Use 60% motor speed to achieve more consistent driving performance.
Low parking exit success rate	Reduce the robot size and test an Ackermann steering system to improve manoeuvrability.
Red + Green pillar avoidance had low success	Reduce the robot body size to improve clearance during directional changes.
Two same-colour pillars caused camera detection issues	Adjust the camera angle to improve pillar detection during movement.

Based on the testing results, the robot size will be reduced to improve manoeuvrability and obstacle clearance. An Ackermann steering system will also be tested to improve the parking exit performance. The camera angle will be adjusted to reduce missed pillar detection when navigating around multiple pillars.

1.2 Power Distribution and Sensor System Architecture

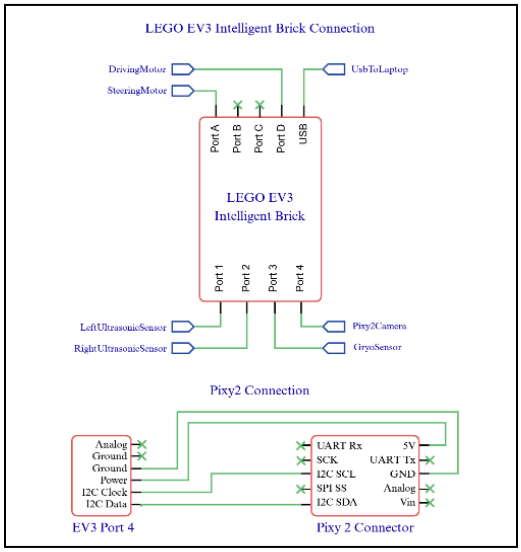
1.2.1 Power Supply and Hardware Connections

The EV3 Brick acts as the central power and control unit. It supplies power to the motors and sensors while processing sensor inputs and controlling the robot's movement. The system uses one Large Motor for driving, one Medium Motor for steering, one Pixy Camera, one Gyro Sensor and two Ultrasonic Sensors.

Power Distribution and Hardware Function Table

Component	Quantity	Power / Function	Purpose
EV3 Brick	1	Main power & control	Supplies power and controls all devices
EV3 Large Motor	1	High-load actuator	Drives the robot
EV3 Medium Motor	1	Steering actuator	Controls steering
Pixy Camera	1	Vision sensor	Detects coloured objects/pillars
Gyro Sensor	1	Orientation sensor	Measures robot rotation
Ultrasonic Sensor	2	Distance sensors	Detect obstacles / field boundaries

Hardware Connection and Communication Architecture



The Generation 1 robot used the EV3 Intelligent Brick as the central control unit. The driving motor, steering motor, ultrasonic sensors, Pixy2 Camera and Gyro Sensor were connected through the EV3 motor and sensor ports. The Pixy2 Camera communicated with the EV3 through the I2C interface. This architecture provided the hardware connections required for both the Open Challenge and Obstacle Challenge programs.

1.2.2 Battery Configuration and Supply Voltage

Battery Configuration

Parameter	Value
Battery Type	AA Battery
Number of Batteries	6
Nominal Voltage per Battery	1.5 V
Total Nominal Voltage	9.0 V

Calculation

Each AA battery provides approximately:

$$V = 1.5V$$

With six batteries connected in series:

$$V_{total} = 6 \times 1.5 V_{total} = 9.0V$$

Therefore, the EV3 system was powered by a nominal 9.0 V battery supply.

1.2.3 Sensor Selection and Functions

Sensor Selection and Functional Comparison

Sensor	Main Function	Advantage	Limitation
Pixy Camera	Detect red/green pillars and orange/blue line	Can identify object colour	Sensitive to lighting and camera angle
Gyro Sensor	Measure rotation/orientation	Provides heading information	Inaccurate reading of the robot's actual heading
Ultrasonic	Detect distance/obstacles	Provides direct distance measurement	Detection can be affected by angle and object shape

Different sensors were used because no single sensor could provide all the information required for navigation. The Pixy Camera was used for colour-based detection of the red and green pillars, while the Gyro Sensor provided rotational information. Two Ultrasonic Sensors provided distance information for obstacle detection. This combination provided complementary sensing, but introduced additional failure points such as camera lighting sensitivity, gyro drift and unreliable ultrasonic readings from angled or narrow objects.

1.2.4 Sensor Placement and Field of View

Sensor Positioning and Placement Rationale

Sensor	Actual Placement	Field Requirement	Justification
Pixy Camera	Upper-middle of robot	Detect red/green pillars and orange/blue	Higher position provides a clearer forward view and reduces obstruction from the chassis
Ultrasonic Sensor (Left)	Front-left	Detect distance on left side	Provides distance information for left-side wall avoidance
Ultrasonic Sensor (Right)	Front-right	Detect distance on right side	Provides distance information for right-side wall avoidance
Gyro Sensor	Centre-bottom of robot	Measure heading	Close to the robot's central axis to reduce the effect of unwanted movement during rotation

Sensor placement was based on the competition field geometry and navigation requirements. The Pixy Camera was mounted at the upper-middle section for a clear forward view and early detection of red and green pillars. Two Ultrasonic Sensors were placed at the front-left and front-right to provide left and right distance information and reduce blind areas. The Gyro Sensor was mounted near the centre-bottom of the chassis to provide a stable reference for robot heading and rotation. This arrangement assigned colour detection, distance detection and heading measurement to the positions best suited for each navigation task.

1.2.5 Sensor Calibration

Sensor Calibration Table

Sensor	Calibration Method	Purpose
Pixy Camera	Calibrate detection of red and green pillars and orange and blue lines under competition lighting. Test detection at the expected distance and viewing angle.	Improve colour recognition and reduce false detection during obstacle avoidance and line detection.
Gyro Sensor	Keep the robot stationary on a level surface during startup to establish a reference heading.	Reduce heading error and improve turning accuracy.

Ultrasonic Sensors	Compare sensor readings with known distances before testing.	Verify distance accuracy and identify abnormal readings.

1.2.5.1 Pixy Camera Colour Recognition

The Pixy Camera was calibrated to detect the target red and green pillars under the competition lighting conditions. The detection was tested from the expected distance and viewing angle to reduce false colour detection.

1.2.5.2 Gyroscope Heading Calibration

The Gyro Sensor was calibrated while the robot was stationary on a level surface. The robot was kept still during startup to establish a stable reference heading before movement.

1.2.5.3 Ultrasonic Distance Calibration

The Ultrasonic Sensors were checked against known distances before testing. The measured values were compared with the actual distances to verify sensor accuracy and identify abnormal readings.

1.2.6 Sensor Edge Case

Sensor	Possible Failure Point	Effect On Robot	Mitigation
Pixy Camera	Lighting / detection angle	Wrong pillar/line detection	Recalibrate and restart camera
Gyro	vibration	Incorrect heading	Calibrate before run and mount rigidly
Ultrasonic	Angled / narrow objects	Incorrect distance	Use two sensors and test detection angle
Wiring	Loose connection	Sensor unavailable	Secure cables and check connections

Sensor reliability was considered because incorrect sensor data could directly affect the robot's navigation decisions. A failure in the Pixy Camera could result in incorrect colour detection, while gyro drift could affect turning accuracy. Ultrasonic readings could also become unreliable when detecting angled or narrow surfaces. Therefore, sensor mounting, calibration and cable connections were checked before testing.

1.2.6.1 Pixy Camera Colour-Detection Accuracy

Colour Target	Total Testing Round	Successful Detection Round	Detection Accuracy
Red Pillar	10	4	40%
Green Pillar	10	3	30%
Orange Line	10	4	40%
Blue Line	10	2	20%

Testing Observation

The Pixy Camera showed higher sensitivity to red and orange colours, with both achieving the highest detection accuracy of 40%. Green showed a lower detection accuracy of 30%, while blue had the lowest at 20%.

Engineering Finding

The Pixy Camera showed inconsistent colour detection, with higher reliability for red and orange than for green and blue, highlighting the need for further calibration to improve detection accuracy.

1.2.6.2 Gyroscope Heading Error and Cumulative Deviation

Testing Round	Target Heading	Actual Heading	Error
1	90°	89.5°	0.5°
2	180°	179°	1.0°
3	270°	268.5°	1.5°
4	360°	358°	2.0°
5	450°	446°	4.0°
6	540°	535.5°	4.5°
7	630°	625°	5.0°
8	720°	714°	6.0°
9	810°	803.5°	6.5°
10	900°	893.5	6.5°
11	990°	983°	7.0°
12	1080°	1072°	8.0°

Average Error

$$\frac{0.5+1+1.5+2+4+4.5+5+6+6.5+6.5+7+8}{12} = 4.04^{\circ}$$

Testing Observation

The Gyro Sensor showed increasing heading error as the cumulative rotation increased. The error increased from 0.5° at 90° to 8.0° at 1080°, with an average error of approximately 4.04° across 12 testing rounds.

Engineering Finding

The results indicate that the Gyro Sensor was generally accurate over smaller rotations but experienced increasing deviation during repeated rotations. This cumulative heading error could affect the robot's turning accuracy during longer navigation sequences.

1.2.6.3 Ultrasonic Distance Accuracy and Detection Range

Actual Distance	Left Reading	Left Error	Right Reading	Right Error
100mm	2550mm	-	2550mm	-
200mm	180mm	20 mm	190mm	10 mm
300mm	290mm	10 mm	290mm	10 mm
400mm	380mm	20 mm	390mm	10 mm
500mm	495mm	5 mm	498mm	2 mm
600mm	595mm	5 mm	597mm	3 mm

2550 mm indicates the sensor returned its maximum detectable distance when the obstacle was too close, rather than providing a valid 100 mm measurement. Therefore, this reading was treated as an invalid reading and was not included in the error calculation.

Testing Observation

At 100 mm, both Ultrasonic Sensors returned 2550 mm, indicating that the obstacle was too close for reliable measurement and the sensors returned their maximum range value. From 200 mm to 600 mm, the readings were much closer to the actual distances, with errors ranging from 2 mm to 20 mm.

Engineering Finding

The testing identified a minimum reliable detection distance as a limitation of the Ultrasonic Sensors. This was important for obstacle avoidance because an obstacle detected within this range could produce an invalid maximum-distance reading and cause incorrect navigation decisions.

1.2.7 Sensor Limitations and Failure Analysis

Based on the sensor testing results, several limitations were identified in the first-generation robot.

Sensor	Testing Result	Issue Identified
Pixy Camera	Accuracy: 20–40%	Inconsistent colour detection
Gyro Sensor	Error increased to 8°	Cumulative heading error
Ultrasonic Sensor	Distance below 100mm reading at 2550mm	Unreliable detection at very short distance

The Pixy Camera showed inconsistent colour detection between different targets. Red and orange were detected more reliably, while green and blue had lower detection accuracy. This could affect the robot's ability to correctly identify pillars and field lines during navigation. The Gyro Sensor showed increasing heading error as the robot rotated repeatedly. The error increased from 0.5° at 90° to 8.0° at 1080°. This cumulative error could affect the accuracy of turning and the robot's overall navigation path. The Ultrasonic Sensors produced an invalid maximum-range reading of 2550 mm when an obstacle was too close at 100 mm. At distances from 200 mm to 600 mm, the readings were more reliable. This showed that very short-distance obstacle detection was a limitation of the sensors.

1.2.8 Sensor Testing Results and Improvement Decisions

Based on the sensor testing results, several changes were decided to improve the reliability of the sensing system.

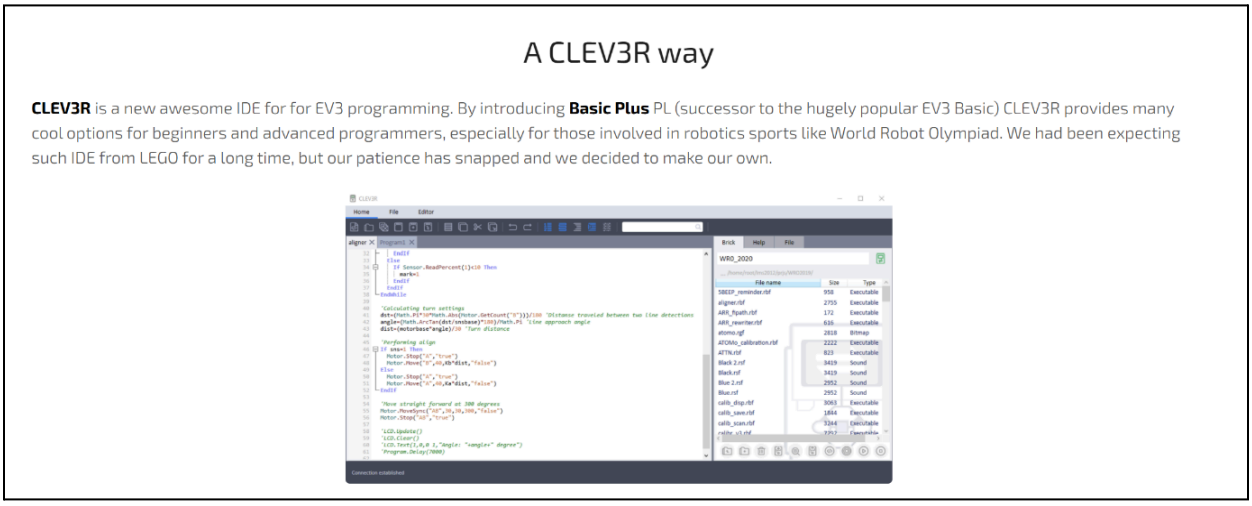
Issue Identified	Testing Result
Inconsistent colour detection by Pixy Camera	Use a Colour Sensor for line detection, while the Pixy Camera is used only to detect red and green pillars.
Increasing Gyro error	Ensure the robot is kept stable and correctly positioned during

during repeated rotation	testing and operation, while avoiding unnecessary shaking that could affect Gyro readings.
Unreliable Ultrasonic readings at short distance	Test both horizontal and vertical sensor placement to determine which position provides more reliable obstacle detection.

The Pixy Camera was assigned mainly to detect the red and green pillars, while a Colour Sensor was selected for line detection to improve detection reliability. For the Gyro Sensor, the robot needs to be kept stable and correctly positioned, with unnecessary shaking minimised to reduce errors during operation. For the Ultrasonic Sensors, both horizontal and vertical mounting positions will be tested to determine the most reliable arrangement for obstacle detection.

1.3 Software Design and Autonomous Navigation

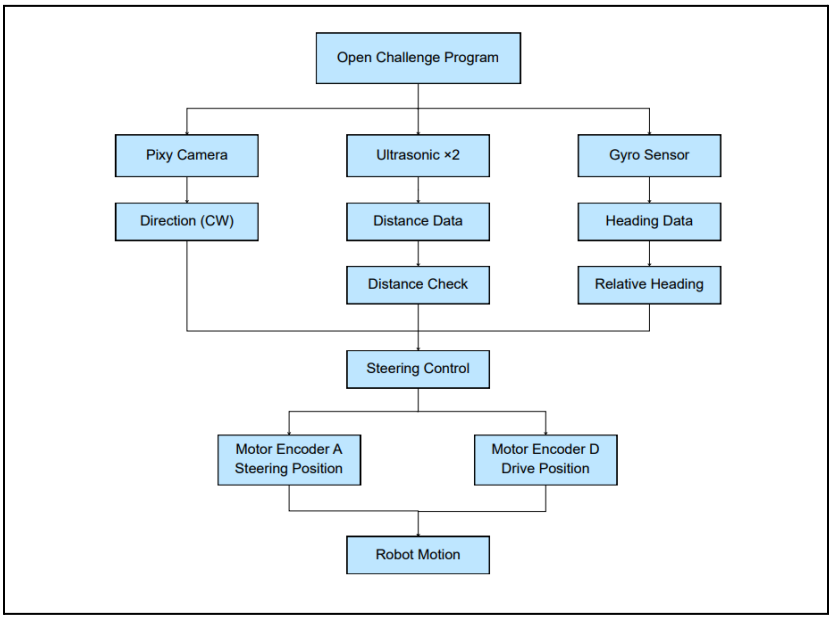
1.3.1 Programming Environment and Development Platform



The robot was programmed using the Clev3r programming application with the LEGO EV3 Brick as the main controller. The program was developed to control motor movement, process sensor inputs and manage the robot's navigation behaviour. The software was modified and tested throughout the different robot generations to improve navigation reliability and obstacle avoidance performance.

1.3.2 Open Challenge Software Architecture and Control Logic

1.3.2.1 Control System Architecture



The Open Challenge program is structured as a sensor-based closed-loop control system. The program uses the Pixy Camera, two Ultrasonic Sensors and a Gyro Sensor as the main sensor inputs. The Pixy Camera is used to determine the initial direction, while the Ultrasonic Sensors provide distance information and the Gyro Sensor provides heading information.The sensor data is processed by the steering and correction routines to determine the required steering response. Motor Encoder A provides steering position feedback, while Motor Encoder D is used to measure the robot's driving movement. The resulting control commands generate the required robot motion.

1.3.2.2 Sensor Input Processing and Feedback Control

The Open Challenge program uses three types of sensor input: the Pixy Camera, two Ultrasonic Sensors and the Gyro Sensor. Each sensor provides different information to the control program. The sensor data is processed continuously through separate software routines and is used to determine the robot's direction, turning heading and steering correction.

1.3.2.2.1 Camera-Based Direction Detection

The Pixy Camera is connected to the EV3 through the I2C interface. The program reads the Pixy data and stores the detected signal in the variable @sig.

```
Sub getPixyLargest
  While 1=1
    values = Sensor.ReadI2CRegisters(4, 1, 80, 2)
    @sig = values[0]
  EndWhile
EndSub
```

The value of CW is then used to determine the direction in which the target heading is updated.

Pixy Signal	Program Action
CW = 1	Target heading = Negatif
CW ≠ 1	Target heading = Positif

The program therefore uses the Pixy Camera primarily to establish the initial turning direction before the repeated navigation sequence begins.

Engineering Purpose:

The Pixy Camera provides an initial directional reference for the navigation sequence. This allows the same program structure to support different turning directions depending on the detected signal. The program does not directly define CW = 1 as a specific colour blue or orange in the code shown. Therefore, the colour-to-direction relationship is not stated here.

1.3.2.2.2 Ultrasonic Distance Control and Steering Correction

Two Ultrasonic Sensors are used to obtain the left and right distance readings. The readings are continuously stored in the variables @l and @r.

```
While 1=1
  @l=Sensor.ReadRawValue(1,0)
  @r=Sensor.ReadRawValue(2,0)
```

The sensor values are used in two different parts of the control system.

A. Navigation Condition

During the main navigation loop, the program waits until the required ultrasonic condition is reached:

```
'Wait Ultrasonic
While @l<2000 And @r<2000
  LCD.Clear()
  LCD.Text(1,0,0,2,@l)
  LCD.Text(1,0,0,2,@r)
EndWhile
```

This allows the program to use the ultrasonic readings as a condition before proceeding to the next turning action.

B. Steering Correction

The Ultrasonic Sensors are also used by the Correction subroutine. When either sensor detects a value below 50, the program applies a fixed steering correction:

```
While 1=1
  If @l<50 Or @r<50 Then
    If @l>@r Then
      turn=35
    Else
      turn=-35
    EndIf
  Else
    turn=relativeHeading*2.0
  EndIf
EndWhile
```

The steering correction therefore depends on the difference between the left and right sensor readings.

Condition	Steering Correction
@l < 50 or @r < 50, and @l > @r	turn = +35
@l < 50 or @r < 50, and @l ≤ @r	turn = −35
Both readings ≥ 50	Heading-based correction

Engineering Purpose:

The two Ultrasonic Sensors provide distance feedback from both sides of the robot. The program uses this information both to determine when the robot can proceed with the navigation sequence and to provide an additional steering correction when the robot is detected to be too close to a boundary or object.

1.3.2.2.3 Gyroscope-Based Heading Control

The Gyro Sensor provides the robot's current heading. The program calculates the difference between the desired target heading and the current Gyro reading:

```
26      relativeHeading=target-Sensor.ReadRawValue(3,0)
```

The calculated value relativeHeading represents the heading error between the target and the

current robot orientation.

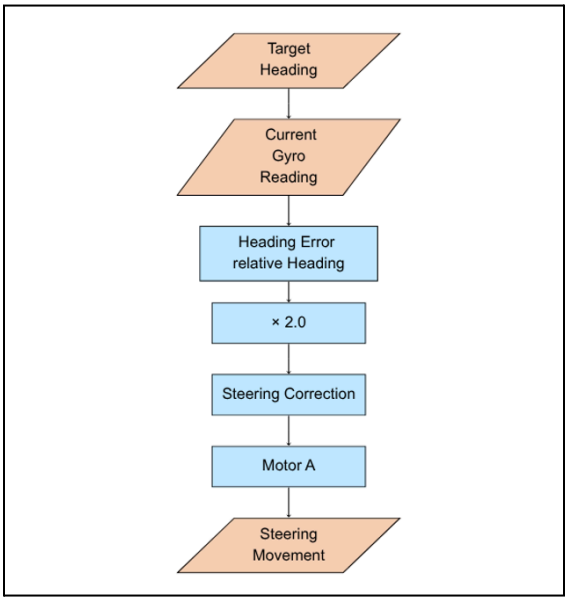
Under normal conditions, this heading error is used in the Correction subroutine:

```
    Else
      turn=relativeHeading*2.0
    EndIf
```

The resulting correction is then included in the steering command:

```
  relativeHeading=target-Sensor.ReadRawValue(3,0)
  correction=-45+turn-Motor.GetCount("A")
  Motor.StartPower("A",correction)
EndWhile
```

The control relationship can therefore be represented as:



Engineering Purpose:

The Gyro Sensor provides feedback on the robot's actual heading during a turn. By comparing the target heading with the measured heading, the program can continuously adjust the steering command instead of relying only on a fixed motor movement.

1.3.2.2.4 Motor Encoder-Based Steering and Distance Control

Motor Encoder A provides steering position feedback, while Motor Encoder D is used to measure the robot's driving movement. These encoder values provide additional feedback to the control system and allow the program to control steering position and travelling distance.

Motor Encoder	Function	Use in Program
Motor A	Steering	Steering position correction
Motor D	Driving	Driving distance / navigation progress

1.3.2.3 Multi-Sensor Integration and Control Decisions

The Open Challenge program combines the Pixy Camera, two Ultrasonic Sensors, Gyro Sensor and motor encoder feedback to control the robot's navigation and turning. The Pixy Camera determines the initial direction, while the Ultrasonic Sensors provide distance information for navigation and steering correction. The Gyro Sensor provides heading feedback by comparing the current heading with the target heading. Motor A and Motor D encoders provide feedback on steering position and driving distance respectively. These inputs are processed by the control logic to generate the required motor commands and robot motion.

Input	Variable	Processing	Main Function
Pixy Camera	@sig	Color detection	Determine initial direction
Left Ultrasonic	@l	Distance comparison	Navigation / steering correction
Right Ultrasonic	@r	Distance comparison	Navigation / steering correction
Gyro Sensor	relativeHeading	target − current heading	Heading control
Motor A Encoder	Motor.GetCount("A")	Steering position feedback	Steering correction
Motor D Encoder	Motor.GetCount("D")	Driving distance feedback	Navigation / distance control

The sensor inputs were assigned different functions so that the robot could use visual information, distance information and heading feedback simultaneously. This provided the first-generation robot with multiple sources of feedback for navigation and turning control.

1.3.2.4 Software Modules and Functional Design

The Open Challenge program is divided into several software modules and functions. Each module is responsible for a specific control task, while the Main Program coordinates the overall navigation sequence. The modular structure allows sensor reading, steering correction and driving-distance control to operate continuously and independently.

Module/Function	Main Function	Main Input	Main Output
getPixyLargest	Continuously reads the Pixy Camera signal	Pixy Camera	@sig
Steering	Controls steering and driving feedback	Ultrasonic, Gyro, Motor A encoder	Motor A steering command
Correction	Determines steering correction	Ultrasonic and Gyro Feedback	turn
waitDegrees()	Controls driving distance	Motor D encoder	Navigation timing
Main Program	Controls the navigation sequence	Pixy, Encoder and Sensor Condition	Target heading and navigation decisions
Multithreading	Runs continuous control modules simultaneously	Software tasks	Concurrent execution
Variables and Feedback	Transfers sensor and control information between modules	Sensor and encoder values	Control parameters

1.3.2.4.1 Pixy Camera Signal Acquisition

The getPixyLargest subroutine continuously reads the Pixy Camera through the I2C interface. The camera data is obtained from Sensor 4 and the detected signal is stored in the variable @sig.

```
Sub getPixyLargest
  While 1=1
    values = Sensor.ReadI2CRegisters(4, 1, 80, 2)
    @sig = values[0]
  EndWhile
EndSub
```

The function runs continuously using an infinite loop. The @sig value is then accessed by the Main Program to determine the robot's initial direction and to detect the required signal change during the final stage.

1.3.2.4.2 Steering Control and Feedback

```
Sub Steering
  Motor.StartPower("A",50)
  Program.Delay(300)
  While Motor.GetSpeed("A")<>0
    'wait
  EndWhile

  Motor.ResetCount("A")
  Motor.StartPower("D",speed)

  While 1=1
    @l=Sensor.ReadRawValue(1,0)
    @r=Sensor.ReadRawValue(2,0)
    relativeHeading=target-Sensor.ReadRawValue(3,0)
    correction=-45+turn-Motor.GetCount("A")
    Motor.StartPower("A",correction)
  EndWhile
EndSub
```

The Steering subroutine is responsible for controlling the steering mechanism and continuously processing feedback from the robot. At the beginning of the function, Motor A is started to initialise the steering mechanism. After a 300 ms delay, the Motor A encoder is reset. Motor D is then started using the defined driving speed.It also reads the Gyro Sensor and calculates the difference between the target heading and current heading.The steering correction is then calculated using the steering motor encoder.The calculated value is sent to Motor A to adjust the steering position. This module therefore provides continuous feedback-based steering control while the robot is moving.

1.3.2.4.3 Distance and Heading Based Steering Correction

```
Sub Correction
  While 1=1
    If @l<50 Or @r<50 Then
      If @l>@r Then
        turn=35
      Else
        turn=-35
      EndIf
    Else
      turn=relativeHeading*2.0
    EndIf
  EndWhile
EndSub
```

The Correction subroutine determines the steering correction based on the Ultrasonic Sensor readings and Gyro heading error.The program first checks whether either Ultrasonic Sensor detects a value below 50. When this condition is met, the program compares the two sensor readings. This produces a fixed steering correction according to the relative distance detected by the two Ultrasonic Sensors. When the ultrasonic condition is not active, the program uses the Gyro heading error. Therefore, the correction module has two operating conditions.

1.3.2.4.4 Encoder-Based Driving Distance Control

```
Function waitDegrees(in number degrees)
  dDegrees=Motor.GetCount("D")
  While Math.Abs(Motor.GetCount("D")-dDegrees)<degrees
    'wait
  EndWhile
EndFunction
```

The waitDegrees() function controls the required driving movement using the encoder feedback from Motor D. It then continuously checks the difference between the current encoder position and the recorded position:

```
While Math.Abs(Motor.GetCount("D") - dDegrees) < degrees
```

The program remains in this loop until the required encoder movement is reached.

For example: waitDegrees(1000)

It requires Motor D to move approximately 1000 encoder degrees before the Main Program continues.

1.3.2.4.5 Main Navigation Sequence and Cycle Management

```
While count<12

  If count=0 Then
    While Motor.GetCount("D")<300 Or @sig<>1 And @sig<>2
      LCD.Clear()
      LCD.Text(1,0,0,2,@sig)
    EndWhile

    CW=@sig

  Else
    waitDegrees(1000)
  EndIf

  'Wait Ultrasonic
  While @l<2000 And @r<2000
    LCD.Clear()
    LCD.Text(1,0,0,2,@l)
    LCD.Text(1,0,0,2,@r)
  EndWhile

  Speaker.Tone(100,440,50)

  If CW=1 Then
    turnDegrees=91
  Else
    turnDegrees=91
  EndIf

  'changing the turning degrees for each turn
  If CW=1 Then
    target=target-turnDegrees
  Else
    target=target+turnDegrees
  EndIf

  count=count+1

EndWhile
```

The Main Program coordinates the overall Open Challenge navigation sequence, including direction selection, navigation cycle counting, target heading control, driving distance and the final turning sequence. At the beginning of the run, the Motor D encoder is reset. During the first navigation cycle, the program waits until Motor D reaches at least 300 encoder degrees or a valid Pixy signal (@sig = 1 or @sig = 2) is detected. The detected Pixy signal is then assigned to CW to determine the robot's travelling direction. For subsequent cycles, waitDegrees(1000) is used to control the driving distance. The program then waits for the required Ultrasonic Sensor condition before updating the target heading by approximately 91°. After each completed cycle, count is increased by one. This navigation sequence is repeated until 12 cycles are completed, after which the program proceeds to the final turning sequence.

1.3.2.4.6 Multithreading and Concurrent Control

```
'Run Task
Thread.Run=getPixyLargest
Thread.Run=Steering
Thread.Run=Correction
```

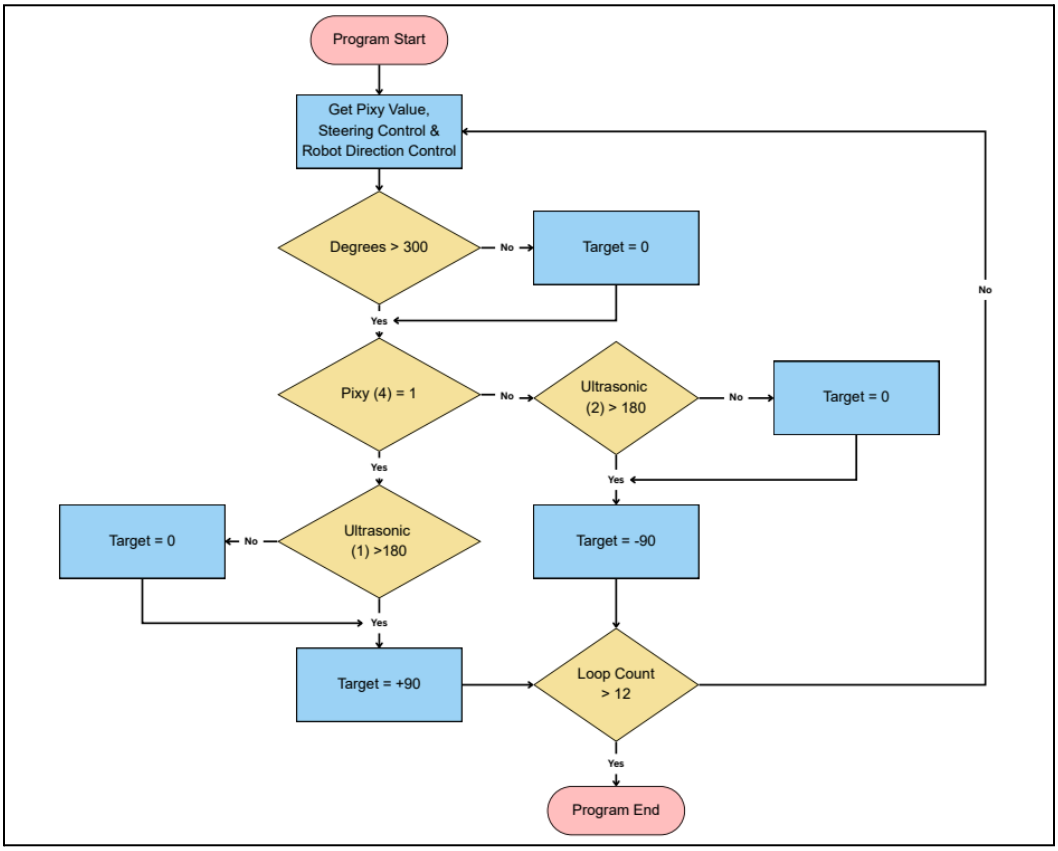
Each thread performs a different control task while the Main Program manages the navigation sequence.

Thread	Module	Purpose
Thread 1	getPixyLargest	Continuously reads Pixy signal
Thread 2	Steering	Continuously controls steering and driving feedback
Thread 3	Correction	Continuously calculates steering correction

The multithreading structure allows sensor reading and steering correction to continue while the Main Program performs higher-level navigation decisions.

1.3.2.5 Open Challenge Navigation State Machine

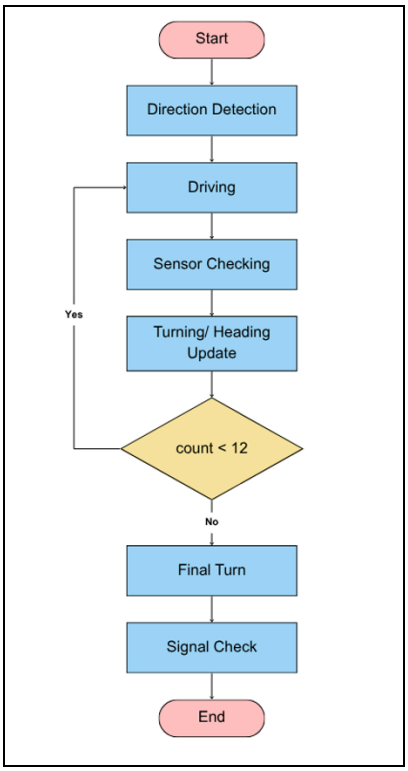
The diagram below shows the state machine for Open Challenge.



The Open Challenge program can be represented as a state-based control sequence. Each state is responsible for a specific navigation task, and the transition to the next state is triggered by a defined sensor, encoder or program condition. This structure provides a predictable navigation sequence and prevents the robot from progressing before the required condition is satisfied.

State	Main Operation	Transition Condition	Rationale
Initialization	Starts the control threads and resets Motor D encoder	Initialization completed	Ensures the sensors, control functions and encoder start from a known condition
Direction Detection	Reads the Pixy Camera signal and determines CW	Valid @sig detected	Establishes the travelling direction before navigation begins
Driving	Motor D drives the robot while steering control operates continuously	Required encoder movement reached	Uses encoder feedback to provide repeatable movement instead of relying only on time
Sensor Checking	Monitors the Ultrasonic Sensor readings	Required ultrasonic condition satisfied	Ensures the robot reaches the required navigation condition before changing its target heading
Turning / Heading Update	Updates target by approximately 91°	Target heading updated	Provides a consistent heading change for each navigation cycle
Repetition	Increases count and repeats the navigation sequence	count < 12	Allows the programmed navigation pattern to be repeated for the required number of cycles
Final Turn	Sets the final target to +1080° or -1080°	Final turning condition reached	Provides the final heading change after the 12 navigation cycles

The state transitions are controlled mainly by sensor readings, motor encoder values and navigation variables rather than fixed delays. For example, the robot remains in the Driving state until the required encoder movement is reached, and it remains in the Sensor Checking state until the required Ultrasonic condition is satisfied.



State Machine Rationale: The state-based structure was selected to divide the Open Challenge into clear and manageable navigation stages. Each transition is based on a specific condition, allowing the robot to wait for the required sensor or encoder feedback before continuing. This reduces uncontrolled transitions and makes the navigation sequence easier to test, tune and modify during development.

1.3.2.6 Navigation Algorithms and Control Rationale

The Open Challenge program uses several simple feedback-based algorithms to control the robot's direction, steering and movement. Each algorithm uses a specific sensor or encoder input and produces a control value for the robot. The algorithms were selected to provide responsive navigation while keeping the control logic simple and suitable for the robot's hardware configuration.

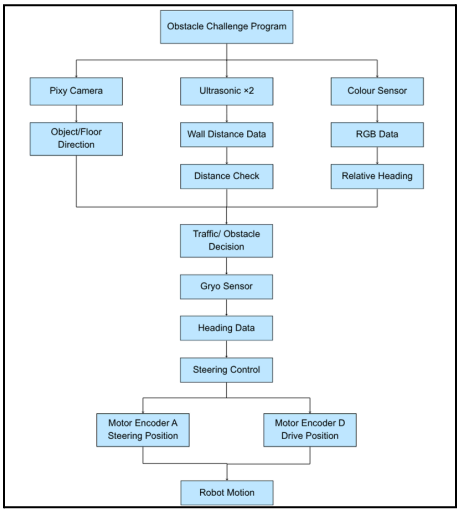
1.3.2.6.1 Camera Vision-Based Direction Selection Algorithm

The Pixy Camera is used to determine the robot's initial travelling direction. The detected Pixy signal is stored in @sig. The Main Program accepts signal 1 or 2 and assigns the detected value to CW.

1.3.3 Obstacle Challenge Software Architecture and Decision Logic

1.3.3.1 Control Architecture and Obstacle Handling

The Obstacle Challenge uses the same basic navigation and steering architecture as the Open Challenge, with additional sensing and decision logic for obstacle and traffic handling. The program processes Pixy Camera, Ultrasonic Sensor, and Gyro Sensor data to determine the robot's steering response.



The additional obstacle-related information is processed by the Traffic and PixySignature routines, while the steering system uses the processed data together with motor encoder feedback to control robot movement.

1.3.3.2 Open Challenge and Obstacle Challenge: Software Comparison

Feature	Open Challenge	Obstacle Challenge
Direction Detection	Pixy Camera	Pixy Camera
Heading Control	Gyro Sensor	Gyro Sensor
Wall Detection	Ultrasonic correction	Ultrasonic correction
Object Detection	No	Pixy Camera detects object signatures
Red/Green Detection	No	Pixy signatures 1 and 2
Stuck Detection	No	Motor D speed monitoring
Reverse Recovery	No	Reverse at -50 power for 500 ms
Steering Decision	Gyro + Ultrasonic	Wall avoidance + traffic following + gyro
Main Navigation	12-turn sequence	12-turn sequence

Compared with the Open Challenge, the Obstacle Challenge adds object detection, wall avoidance, traffic following and stuck-recovery logic. The basic navigation, gyro correction and encoder-based movement remain similar.

1.3.3.3 Obstacle Challenge Software Modules and Responsibilities

Module / Function	Main Purpose
getRGB()	Reads RGB values from the Pixy Camera
getLargest()	Obtains the largest detected Pixy object and its position/size
getSignature()	Obtains the position and size of a specific Pixy signature
PixySignature	Continuously processes floor and red/green object signatures
PixyWall	Continuously reads front, left and right RGB values
Traffic	Handles wall avoidance, traffic following, heading correction and stuck recovery

1.3.3.3.1 RGB Data Acquisition from the Camera

```
Function getRGB(in number port, in number xin, in number yin,
  out number w)
  writeBuffer[0]=94
  writeBuffer[1]=Math.Remainder((xin*0.80952320952)+128,256)-128
  writeBuffer[2]=Math.Remainder((yin*1.23188405797)+128,256)-128
  writeBuffer[3]=0
  values=Sensor.CommunicateI2C(port,1,4,3,writeBuffer)
  r=values[0]
  g=values[1]
  b=values[2]
  w=r+g+b
EndFunction
```

The getRGB() function communicates with the Colour Sensor and obtains the red, green and blue values at specified coordinates. The combined RGB value is stored as w for further processing.

1.3.3.3.2 Camera-Based Object Detection and Position Extraction

```
Function getLargest(in number port,out number sig, out number
  x, out number y, out number w, out number h)
  values = Sensor.ReadI2CRegisters(port, 1, 80, 6)
  sig = values[0]
  x = values[2]
  y = values[3]
  w = values[4]
  h = values[5]
EndFunction
```

The getLargest() function reads Pixy Camera data and returns the detected object's signature, position and size. The parameters x, y, w and h are used by the program to determine the object's location and relative size.

1.3.3.3.3 Object Signature Identification and Position Tracking

```
Function getSignature(in number port, in number sig, out number
  x, out number y, out number w, out number h)
  address = 80 + sig
  values = Sensor.ReadI2CRegisters(port, 1, address, 5)
  'num_of_matches = values[0]
  x = values[1]
  y = values[2]
  w = values[3]
  h = values[4]
EndFunction
```

The getSignature() function retrieves the position and size of a specific Pixy signature. It is used to obtain separate data for the detected red and green objects.

1.3.3.3.4 Continuous Floor and Object Detection

```
Sub PixySignature
  While 1=1
    floorValues = Sensor.ReadI2CRegisters(4, 1, 80, 6)
    @floorSig = floorValues[0]
    getLargest(1,@sig,@x,@y,@w,@h)
    getSignature(1,1,@x1,@y1,@w1,@h1)
    getSignature(1,2,@x2,@y2,@w2,@h2)
  EndWhile
EndSub
```

PixySignature continuously reads the floor signature and the detected red and green object information. This allows the main program and traffic logic to access updated object data during navigation.

1.3.3.3.5 Multi-Position Colour Sensing for Wall Detection

```
Sub PixyWall
  While 1=1
    getRGB(2,200,40,@f)
    getRGB(2,50,128,@l)
    getRGB(2,250,128,@r)
  EndWhile
EndSub
```

PixyWall continuously reads RGB values from the front, left and right positions of the Pixy Camera. These readings provide additional information for wall and obstacle handling.

1.3.3.3.6 Integrated Traffic Handling and Recovery Control

```
If trafficDistance<260 And logic=1 Then

  'check Green or Red to follow traffic
  If (@w1*@h1)>@w2*@h2 Then

    If @x1<green Or @y1>200 Then
      'Follow Green Traffic
      turn=((green-@x1)*@y1*0.5)*0.02
    Else
      turn=relativeHeading*1
    EndIf

  Else

    If @x2>red Or @y2>200 Then
      'Follow Red Traffic
      turn=((red-@x2)*@y2*0.5)*0.02
    Else
      turn=relativeHeading*0.8
    EndIf

  EndIf
```

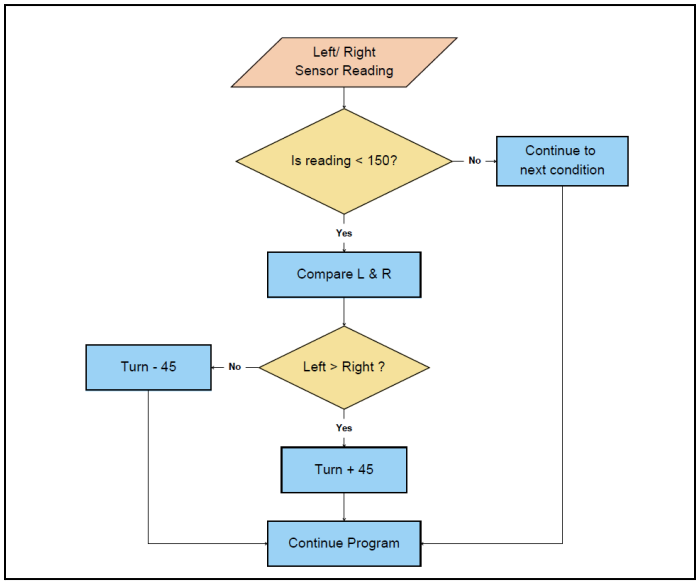
The Traffic subroutine is the main obstacle-handling module. It calculates the detected traffic distance, checks the left and right wall distances, determines whether to follow a red or green object, and calculates the required steering correction. It also detects when the robot may be stuck and temporarily reverses the driving motor to recover.

1.3.3.4 Obstacle Detection, Avoidance and Recovery Algorithms

The Obstacle Challenge extends the basic navigation system used in the Open Challenge with additional algorithms for wall avoidance, traffic distance detection, object avoidance and stuck recovery. These four algorithms provide the main obstacle-specific decision-making of the Generation 1 program. Wall avoidance and object avoidance handle environmental obstacles, while traffic distance provides additional object information and stuck recovery improves the robot's ability to recover from unsuccessful movement.

1.3.3.4.1 Ultrasonic-Based Wall Avoidance Algorithm

The program checks the left and right sensor readings to determine whether the robot is close to a wall. When either reading is below the defined threshold of 150, the program enters the wall avoidance logic.



The steering direction is selected by comparing the two readings. If the left reading is greater than the right reading, the program applies a positive steering correction. Otherwise, it applies a negative correction. This algorithm allows the robot to react to a nearby wall by changing its steering direction according to the relative sensor readings.

Rationale: This algorithm was selected to provide a simple and fast response when the robot approaches a wall. By comparing the left and right sensor values, the robot can determine the safer direction and apply a steering correction. This approach is suitable for real-time obstacle avoidance because it requires low computational complexity and can respond quickly to changes in the environment.

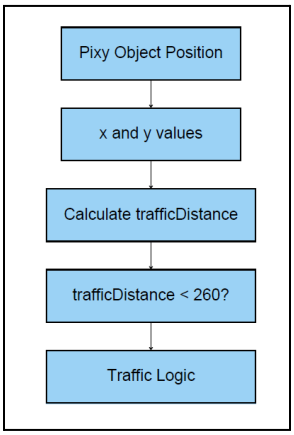
1.3.3.4.2 Camera-Based Relative Object Proximity Estimation

The program uses the position of the detected object from the Pixy Camera to estimate its relative distance. The object position is represented by x and y.

The program calculates:

$$trafficDistance = \sqrt{(x - 128)^2 + (y - 300)^2}$$

The calculated value is then compared with a threshold of 260.



This calculation provides a simple way for the robot to determine whether a detected object is close enough to require additional obstacle-handling logic.

Rationale: The traffic distance calculation was used to determine whether a detected object was close enough to require an avoidance response. The distance is estimated from the detected object's position relative to the camera reference point. A threshold was introduced to prevent the robot from reacting to objects that were too far away, reducing unnecessary steering corrections.

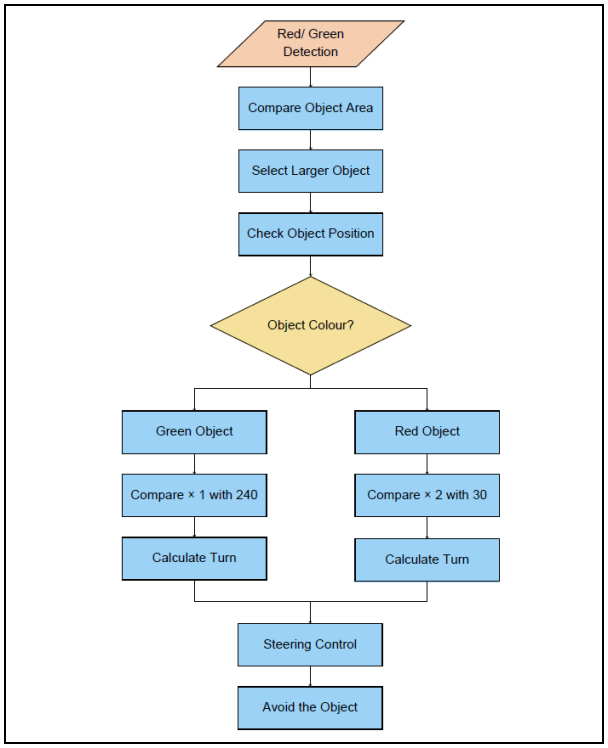
1.3.3.4.3 Colour and Position-Based Object Avoidance Algorithm

The robot uses the Pixy Camera to detect red and green objects and determine their positions in the camera view. The program obtains the object's signature, horizontal position (x), vertical position (y), width (w) and height (h).

First, the program compares the detected object areas:

$$A_{green} = w_1 \times h_1 A_{red} = w_2 \times h_2$$

The object with the larger detected area is selected for the avoidance decision.



For the green object, the program uses x1 and y1 to determine its position relative to the green reference position (green = 240). The steering correction is calculated as:

$$turn = \left(\left(green - x_1\right) \times y_1 \times 0.5\right) \times 0.02$$

For the red object, the program uses x2 and y2 relative to the red reference position (red = 30):

$$turn = \left(\left(red - x_2\right) \times y_2 \times 0.5\right) \times 0.02$$

The calculated turn value is then used by the steering control to change the robot's direction.

Rationale: This algorithm was selected to distinguish between red and green objects and determine an appropriate avoidance direction. The program compares the detected object areas to identify the dominant object and uses its position to calculate the steering correction. This allows the robot to react dynamically to different object positions instead of using a fixed steering command.

1.3.3.4.4 Motor-Speed-Based Stuck Detection and Reverse Recovery

The program also includes a basic recovery algorithm to detect when the robot may be stuck. The driving motor speed is continuously monitored.

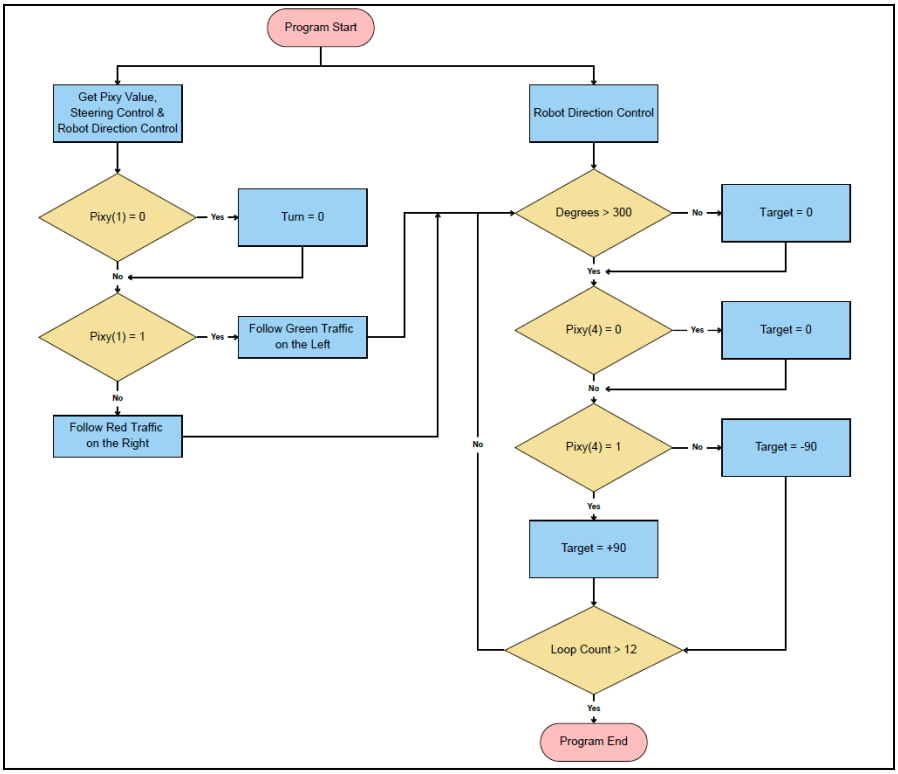
```
If (Motor.GetSpeed("D")<20 And Time.GetS()>1000) Then
  Speaker.Tone(100,240,50)
  speed=-50
  Program.Delay(500)
  speed=Mainspeed
  Time.Reset()
```

When the driving motor speed remains below 20 after the defined time condition, the program temporarily sets the driving speed to −50 and reverses the robot for 500 ms. The robot then returns to its normal Mainspeed. This recovery mechanism prevents the robot from remaining stationary when an obstacle or other condition prevents normal forward movement.

Rationale: A stuck detection and recovery algorithm was implemented to handle situations where the robot could no longer move forward. When the driving motor speed remains below the defined threshold for a certain period, the robot temporarily reverses to escape the stuck position before returning to its normal driving speed. This improves reliability by providing an automatic recovery response without requiring manual intervention.

1.3.3.5 Obstacle Challenge Navigation State Machine

The diagram below shows the state machine for the Obstacle Challenge.



The state machine allows the robot to follow the planned route while responding to walls, coloured objects and movement conditions. The main navigation loop is repeated until the required number of navigation cycles is completed, after which the program performs the final turn and completes the sequence.

State	Main Operation	Transition Condition	Rationale
Initialization	Starts control threads and initializes variables	Initialization completed	Establishes a known starting condition
Direction Detection	Reads the Pixy Camera floor signal to determine CW/CCW	Valid floor signal detected	Establishes the moving direction
Driving	Motor D drives while steering control operates continuously	Required encoder movement reached	Provides repeatable movement using encoder feedback
Obstacle Detection	Checks wall distance and detected object information	Obstacle/traffic condition detected	Allows the robot to respond to environmental obstacles
Obstacle Decision	Selects wall avoidance or red/green object avoidance response	Appropriate steering decision determined	Determines the required steering response
Steering	Applies the calculated steering correction while driving	Navigation condition reached	Moves the robot while responding to obstacle information
Repetition	Increases count and repeats the navigation sequence	count < 12	Completes the required navigation cycles
Final Turn	Changes the final target heading after the navigation cycles	Final condition reached	Completes the final navigation movement

The state-based structure divides the Obstacle Challenge into clear navigation and obstacle-handling stages. Sensor and encoder conditions are used to control transitions between states, allowing the robot to respond to obstacles while maintaining the planned

navigation sequence. This structure also makes the program easier to test and modify during development

1.3.3.6 Obstacle Avoidance Algorithm Testing and Performance Evaluation

Test Item	Test Condition	Trials	Successful	Success Rate	Result
Wall Avoidance	Wall less than 150mm	5	4	80%	The robot successfully avoided the wall in most tests, but occasional incorrect steering caused a collision.
Traffic Detection	Different object distances	5	3	60%	The robot detected objects at some distances, but detection was inconsistent when the object position or distance changed.
Object Avoidance	Red / Green objects	5	2	40%	The robot could detect and avoid some red and green objects, but the steering response was not consistent.
Stuck Recovery	Robot intentionally blocked	5	2	40%	The robot recovered from some blocked positions, but the recovery response was not reliable in every situation.
Overall Obstacle Challenge	Full course	5	1	20%	The complete challenge had low reliability, showing that further software tuning and integration improvements were required.

The testing results showed that individual obstacle-handling functions performed better than the complete Obstacle Challenge. Wall avoidance achieved the highest success rate at 80%, while traffic detection achieved 60%. Object avoidance and stuck recovery achieved 40%. However, the overall success rate was only 20%, indicating that the main limitation was not only individual algorithms but also the interaction between different control functions during operation.

1.3.3.7 Edge-Case Handling and Navigation Failure Analysis

The Obstacle Challenge program includes several conditional checks and control loops to handle situations where obstacles are detected or the robot cannot move as expected. Instead of immediately continuing to the next navigation stage, the program applies the corresponding avoidance or recovery response based on the detected condition.

Edge Case	Program Response	Purpose
Ultrasonic sensor detects the wall at a very close distance	Apply the wall avoidance steering correction based on the sensor reading	The robot may detect the wall too late, causing the steering response to be delayed and resulting in a collision with the wall
Object is detected too late for avoidance	Apply the calculated steering correction after the object is detected	The robot may not have enough time or distance to complete the avoidance manoeuvre, resulting in a late response or collision

These edge cases were identified during software testing of the Obstacle Challenge. The testing showed that the robot's response depends on the timing of sensor detection. When the wall or object is detected at a very close distance, the robot has insufficient time to complete the required steering correction. This can result in a delayed avoidance response or collision. These findings highlighted the importance of improving detection timing and avoidance response during further software iteration.

1.4 Generation 1 Evaluation and Design Improvements

Following Generation 1 testing, key limitations were identified in robot size, steering performance, ultrasonic and colour detection, and sensor integration. These findings guided the design improvements introduced in Generation 2.

No.	Aspect	Issue Identified	Improvement Decision
1	Robot Size & Camera Position	The robot was relatively large due to the use of a Large Motor. The larger structure reduced manoeuvrability during parking and pillar avoidance. The camera position also provided a limited field of view.	Redesign the robot using two Medium Motors and smaller wheels to reduce the overall size. Move the camera further backwards to provide a wider field of view.
2	Steering System	The existing steering system had a limited turning angle, making tight turns more difficult.	Test a geared-down steering system with Ackermann steering to improve the turning angle and manoeuvrability.
3	Ultrasonic Detection	The ultrasonic sensor could produce missed readings when the robot was very close to the wall, resulting in a late avoidance response.	Change the ultrasonic sensor to a horizontal orientation and test whether close-range detection can be improved.
4	Floor Colour Detection & Sensor Ports	The camera frequently missed the four floor colours. The limited number of EV3 sensor ports also restricted the number of sensors that could be used.	Add a Colour Sensor for orange and blue floor detection. Use a multiplexer to increase the available sensor ports
5	Camera Function	Although the camera was unreliable for detecting all four floor colours, it remained useful for detecting red, green and magenta.	Keep the camera for red, green and magenta detection, while the Colour Sensor is used for orange and blue floor detection.

Based on Generation 1 testing, Generation 2 focused on enhancing manoeuvrability, steering performance, sensor reliability and integration through a more compact design featuring two Medium Motors, smaller wheels and Ackermann steering. The Pixy Camera and Colour Sensor were assigned separate colour-detection functions, while a multiplexer was introduced to overcome the EV3's limited sensor ports.

2.0 Generation 2: Design Refinement and Engineering Validation

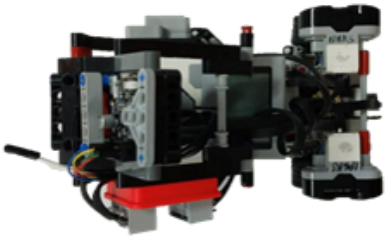
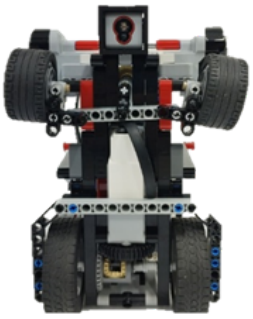
2.1 Overview

Based on the issues identified in Generation 1, the robot was redesigned to improve manoeuvrability, steering, sensor reliability and software performance. Generation 2 introduced a smaller robot structure, two Medium Motors, a geared-down Ackermann steering system, improved sensor placement and additional sensors using a multiplexer. The software was also redesigned to support the new hardware and improve navigation and obstacle avoidance. These changes were then tested to determine whether the problems identified in Generation 1 had been improved.

2.2 Mechanical Redesign and Mobility Improvements

The mechanical redesign from Generation 1 to Generation 2 is shown in the table below.

Gen 1		Gen 2
Front View		Front View
Back View		Back View

Left View	→	Left View
Right View	→	Right View
Top View	→	 Top View
Bottom View	→	 Bottom View

2.2.1 Chassis Configuration and Wheel Selection

Generation 2 was redesigned with a smaller and more compact structure and two Medium Motors to improve manoeuvrability in tight areas. For the steering system, 56145 rims with 44309 tyres were used, while 32019 wheels with 86652 rims were used for driving.

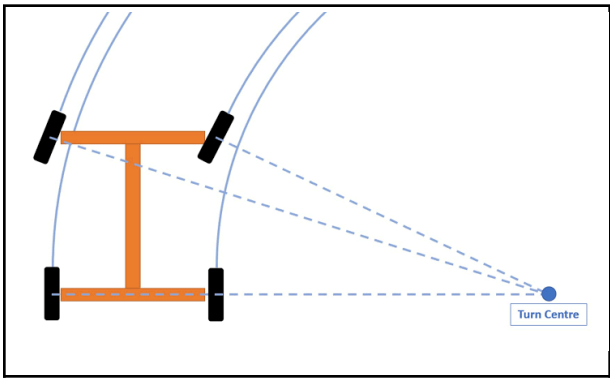
Generation 1 - Issue	Generation 2 - Improvement
Large robot structure	Smaller and more compact structure
Difficult manoeuvrability	Smaller overall size
Large wheels	Smaller wheels

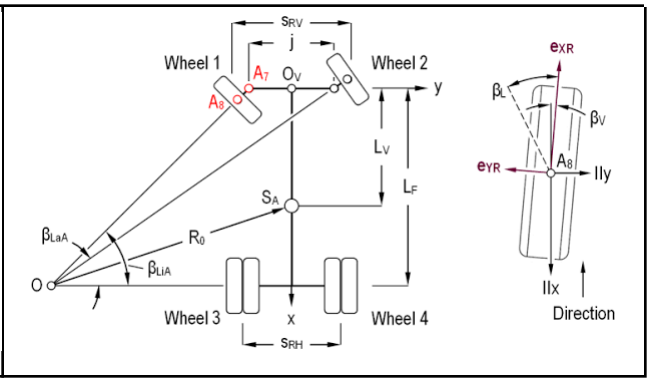
2.3 Steering System Redesign and Performance Evaluation

2.3.1 Generation 1 Steering Limitations

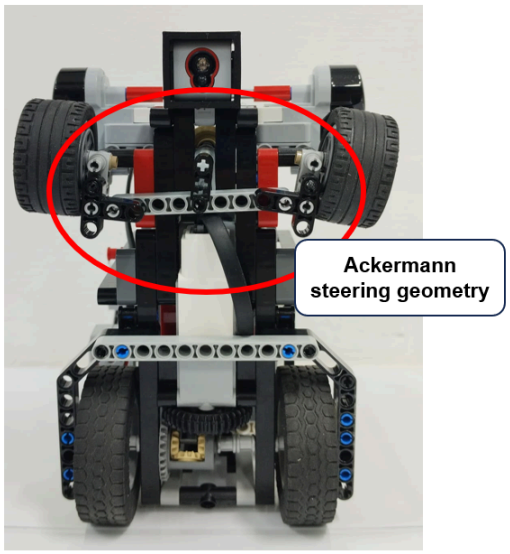
In Generation 1, the steering system had a limited turning angle, which made it difficult for the robot to perform tight turns during parking exit and obstacle avoidance. The steering response was also limited by the original mechanical structure.

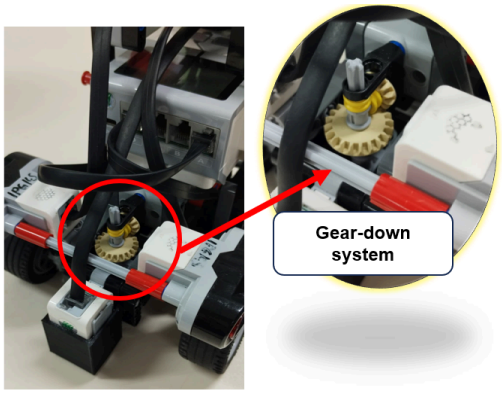
Solution: To improve the steering performance, Generation 2 used a geared-down steering system with Ackermann steering geometry. The geared-down mechanism was used to provide better steering control, while the Ackermann geometry allowed the inner and outer wheels to turn at different angles during a turn.





2.3.2 Geared-Down Transmission and Ackermann Steering Geometry





The steering system was redesigned to improve steering control and manoeuvrability. A geared-down mechanism and Ackermann steering geometry were introduced based on the limitations observed in Generation 1.

2.3.3 Steering Design Concept and Engineering Rationale

Design Issue in Gen 1	Design Concept in Gen 2	Rasionale
Steering response was difficult to control during tight turns	12T → 20T gear reduction	Reduce steering speed and increase available torque
Limited manoeuvrability during tight turns	Ackermann steering	Allow the inner and outer wheels to turn at different angles
Steering motor movement was transferred directly to the steering mechanism	Geared-down transmission	Provide more controlled steering movement
Robot needed better turning performance	Combined gear reduction + Ackermann geometry	Improve steering control and turning behaviour

The steering concept was selected based on the Generation 1 testing results. The main objective was not to increase the maximum steering angle directly, but to obtain more controlled steering movement and sufficient torque for tight turns.

2.3.4 Steering Performance Testing and Design Selection

2.3.4.1 Steering Angle Response and Accuracy

Motor Input	Gen 1 Target	Gen 1 Actual	Gen 1 Error	Gen 2 Theoretical*	Gen 2 Actual	Gen 2 Error
10°	10°	12°	2°	6°	7°	1°
20°	20°	21°	1°	12°	11°	1°
30°	30°	33°	3°	18°	19°	1°
40°	40°	39°	1°	24°	26°	2°
Average Error	-	-	1.75°	-	-	1.25°

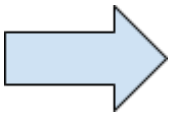
Gen 2 theoretical angle is calculated using the 12T → 20T gear reduction:

Rasionale: The testing showed that Generation 2 had a lower average angle error of 1.25°, compared with 1.75° for Generation 1. This indicates that the geared-down system produced a more controlled and consistent output angle based on its theoretical gear ratio. However, the 12T to 20T gear reduction changed the relationship between the motor input and the actual steering angle. Each steering command therefore required a 1.67:1 conversion and additional calibration. Although Generation 2 provided slightly better angle accuracy, the additional calculation increased the complexity of programming and tuning. Therefore, the direct steering system was selected for further development because it provided a simpler motor-to-wheel relationship and reduced the need for repeated angle conversion and calibration.

2.4 Sensor Placement and Detection Performance

Based on the limitations identified in Generation 1, the sensor placement was modified in Generation 2 to improve detection performance. The camera was moved further backwards and positioned at a lower height to provide a wider field of view. The ultrasonic sensors were also tested in both vertical and horizontal orientations to determine a more suitable position for obstacle detection.

2.4.1 Camera Position and Field-of-View Improvement

Gen 1		Gen 2
		

In Generation 1, the camera position provided a more limited view of the field. In Generation 2, the camera was moved further backwards and placed at a lower height. This increased the field of view and allowed the camera to capture a wider area in front of the robot.

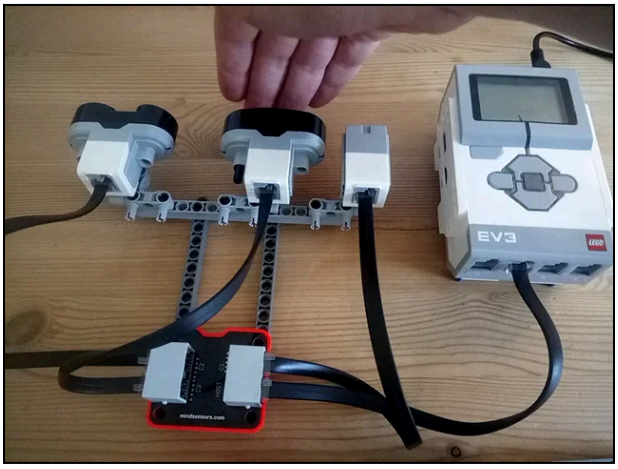
Table Comparison of Camera Placement Between Generation 1 and Generation 2:

Aspect	Generation 1	Generation 2	Rationale
Camera Position	Further forward	Move backwards	Increase the effective field of view and provide earlier visual information for navigation decisions.
Camera Height	Higher	Lower	Increase the coverage of the field surface within the camera view and improve detection of floor features.

2.4.2 Ultrasonic Sensor Orientation Experiment

In Generation 1, the ultrasonic sensors were mounted horizontally. In Generation 2, a vertical orientation was tested to investigate whether changing the sensor orientation could improve obstacle detection. The result of this test was later used to determine the sensor orientation for the following generations.

2.5 Sensor Architecture and Multi-Sensor Integration



Generation 2 introduced a multiplexer to overcome the limited number of EV3 sensor ports. With the multiplexer, the robot could support six sensing devices: one Colour Sensor, two Pixy Cameras, two Ultrasonic Sensors and one Gyro Sensor. The sensors were assigned different functions to improve the robot's ability to detect the field, obstacles and parking area.

2.5.1 Sensor Configuration and Functional Allocation

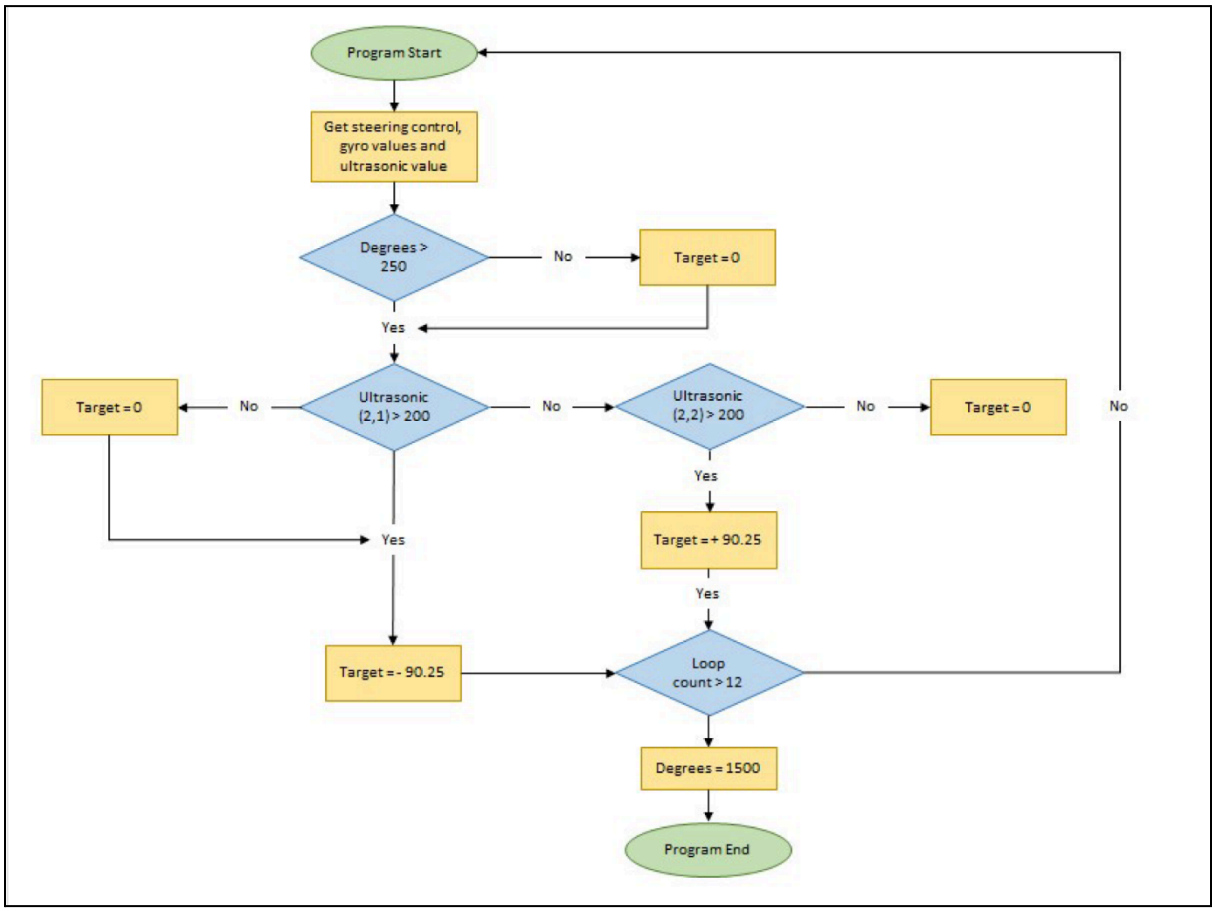
Sensor	Quantity	Function	Engineering Rasionale
Colour Sensor	1	Detect floor colours	Provide a dedicated sensor for floor colour detection.
Pixy Camera	2	Detect parking area and green& red pillar	Provide visual information for object avoidance and parking manoeuvre.
Ultrasonic Sensor	2	Detect obstacles/walls	Provide distance information from both sides of the robot.
Gyro Sensor	1	Measure heading	Provide orientation feedback for steering and turning.

Multiplexer Integration and Sensor Selection Rationale: The sensor architecture was designed based on the different sensing requirements of the robot. The Colour Sensor was assigned to floor colour detection, the front Pixy Camera to green and red pillar detection, and the rear Pixy Camera to parking area detection. The Ultrasonic Sensors provided obstacle information, while the Gyro Sensor provided heading feedback. The multiplexer was therefore selected to support all six sensing functions within the EV3 system.

2.6 Software Redesign and Control Architecture

2.6.1 Open Challenge

2.6.1.1 Software Architecture and Control Flow



The program starts by initializing the sensors, variables and motor controls. During operation, sensor data from the Gyro Sensor and distance sensors are processed to determine the required steering correction. The steering motor then responds to the calculated correction, while the driving motor encoder is used to control the travelling distance. This process is repeated for each navigation loop until the required number of loops is completed.

2.6.1.2 Software Redesign and Feedback Control Integration

Software Aspect	Generation 2 Implementation	Purpose
Steering Control	Gyro + Wall Correction	Improve steering accuracy
Gyro Correction	Proportional + change in error	Correct heading deviation
Wall Correction	Wall distance error + change in error	Maintain suitable wall distance
Steering Feedback	Motor A encoder	Match steering motor position with calculated turn
Movement Control	Motor D encoder	Control travelled distance
Direction Detection	Left/right distance sensor	Determine CW or CCW
Navigation Loop	12 loops	Complete the required navigation sequence

Based on the problems identified during Generation 1 testing, the software control system was redesigned to improve steering stability and navigation accuracy. The main changes focused on adding multiple feedback sources instead of relying mainly on heading correction.

The redesigned system consisted of the following:

- **Gyro Correction –** calculates the difference between the target heading and the current heading to correct the robot's orientation.
- **Wall Correction –** uses the detected wall distance to adjust the steering when the robot is navigating close to the wall.
- **Steering Motor Feedback –** uses the Motor A encoder to compare the calculated steering command with the actual steering position.
- **Driving Motor Feedback –** uses the Motor D encoder to control the travelling distance.
- **Navigation Loop –** divides the navigation into repeated sections, with the target heading updated after each approximately 90° movement.

The redesign was justified by the Generation 1 testing results, where the robot experienced difficulty maintaining a stable path during turning and obstacle navigation. The addition of wall-distance feedback allowed the controller to consider the robot's position relative to the wall, while gyro feedback maintained the required heading. Steering motor feedback was included to improve the accuracy of the steering response.

Rasionale: A combined feedback approach was selected for Generation 2 because steering control depended on more than the robot's heading alone. By combining gyro, wall-distance and motor-position feedback, the system could respond to both orientation and position errors during navigation. This provided a more controlled steering system for further testing and development.

2.6.1.3 Navigation and Steering Control Algorithms

The Generation 2 steering system uses feedback from the Gyro Sensor, wall-distance sensors and steering motor encoder to determine the required steering response. The algorithm was designed to consider both the robot's heading and its position relative to the wall.

2.6.1.3.1 Heading Error Calculation

The heading error is calculated using the target heading and the current Gyro Sensor reading:

```
40    relativeHeading = Sensor.ReadRawValue(3,0)*-1 + target
41
```

This represents the difference between the required heading and the current heading of the robot. The calculated error is then used to determine the steering correction.

2.6.1.3.2 Gyro-Based Heading Correction

The Gyro Sensor correction is calculated using the current heading error and the change in heading error:

```
55    'Gyro Correction
56    gyroCorrection = relativeHeading * 0.8 + (relativeHeading-gyroLastError) * 10
57
```

The current heading error provides the main correction, while the change in error adjusts the steering response when the heading error changes.

2.6.1.3.3 Wall-Distance Correction

Wall correction is used when the robot is navigating close to a wall. The measured wall distance is compared with a reference distance of 180 mm:

```
'Wall Correction
wallError= (180 - Wall) * cw
wallCorrection = wallError * 0.07 + (wallError-wallLastError) * 0.5
```

The wall error represents the difference between the measured wall distance and the reference distance. This correction helps the robot adjust its steering according to its position relative to the wall.

2.6.1.3.4 Integrated Steering Correction and Motor Feedback

The final steering command combines the two feedback corrections:

```
'Steering Correction
turn = gyroCorrection + wallCorrection
```

This allows the robot to consider both its orientation and position relative to the wall when calculating the steering response.

The steering motor encoder is then used to compare the calculated steering command with the actual steering motor position:

```
'Power To Medium Motor
steeringPower = (turn - Motor.GetCount("A")) * 1.5
Motor.StartPower("A", steeringPower)
```

This provides additional feedback to control the steering motor movement.

Rationale: The combined feedback approach was selected to improve steering control compared with the simpler heading-based correction used in Generation 1. Gyro feedback maintains the required heading, while wall-distance feedback helps control the robot's position relative to the wall. The steering motor encoder provides additional feedback on the actual steering position. This combination was intended to provide a more stable and controlled steering response during navigation.

2.6.1.3.5 Navigation Direction Detection and Heading Updates

The Generation 2 navigation system determines the travelling direction using the left and right distance sensor readings. At the beginning of the navigation sequence, the program compares the sensor values to determine whether the robot should travel clockwise (CW) or counter-clockwise (CCW).

```
If rightDistance>2000 Then
  cw=1
Else
  cw=-1
EndIf
```

A value of cw = 1 represents clockwise movement, while cw = -1 represents counter-clockwise movement. This allows the program to automatically select the navigation direction based on the detected field condition. After each navigation loop, the target heading is updated according to the selected direction:

```
target = target - 90.25 * cw
loopCount = loopCount + 1
```

The target heading is therefore changed by approximately 90.25° per navigation loop. This provides a new heading reference for the steering control system before the robot enters the next section of the course.The navigation process is repeated for 12 loops, with the target heading updated after each loop. During movement, the driving motor encoder is also used to control the travelling distance.

Rationale: The direction and navigation system was designed to provide a consistent heading reference throughout the course. Automatic direction detection reduces the need for a fixed starting direction, while the approximately 90.25° heading update allows the robot to maintain a structured navigation sequence through repeated turns.

2.6.1.4 Open Challenge Performance Evaluation

After implementing the redesigned software and mechanical changes, the Generation 2 robot was tested in the Open Challenge to evaluate its overall navigation performance. The testing focused on completion time and consistency and was compared with the Generation 1 results. This comparison was used to evaluate the trade-off between driving speed and navigation stability.

Comparative Testing and Navigation Reliability Analysis

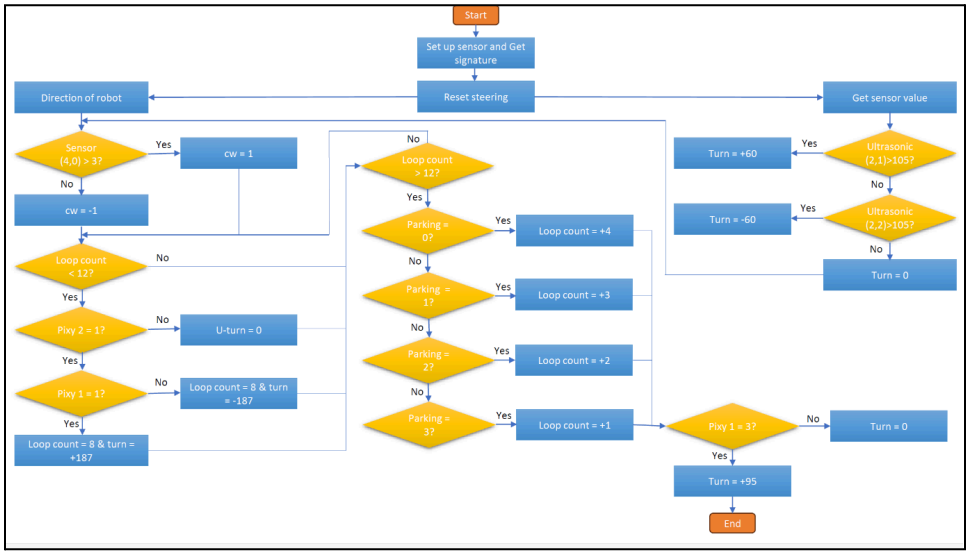
Robot	Trial Speed (%)	Total Rounds	Average Completion Time (s)	Successful Rounds	Success Rate (%)
Generation 1	100	10	18	2	20
Generation 2	100	10	22	7	70

Testing Result and Analysis: Both generations were tested at 100% trial speed with 10 testing rounds to provide a direct comparison. Generation 1 achieved a faster average completion time of 18 seconds, while Generation 2 recorded 22 seconds. However, Generation 2 achieved 7 successful rounds out of 10, giving a 70% success rate, compared with only 2 successful rounds out of 10 (20%) for Generation 1. This shows that although Generation 1 was faster, its navigation performance was less consistent. The 4-second increase in average completion time from Generation 1 to Generation 2 represents a trade-off for the improvement in success rate from 20% to 70%. The result indicates that the Generation 2 design provided more controlled and reliable navigation at the same trial speed.

Engineering Decision: The testing results supported the decision to prioritise stability over maximum speed in Generation 2. Although Generation 2 required more time to complete the Open Challenge, its higher success rate demonstrated improved consistency and reliability compared with Generation 1.

2.6.2 Obstacle Challenge

2.6.2.1 Software Architecture and Obstacle Navigation Control



The Generation 2 Obstacle Challenge software was developed as an integrated control system combining sensor input, navigation control, obstacle detection, steering correction and parking control. The program continuously processes information from the distance sensors, Gyro Sensor and Pixy Camera to determine the robot's movement and steering response.

2.6.2.2 Software Redesign and Multi-Sensor Integration

Based on the limitations identified during the Generation 1 Obstacle Challenge testing, the Generation 2 software was redesigned to provide more integrated control of navigation and obstacle avoidance. The redesign focused on improving the robot's ability to respond to walls, coloured pillars, field markers and heading deviation during movement.

Software Aspect	Generation 2 Implementation	Purpose
Steering Control	Gyro + Wall + Pillar Correction	Improve steering response during obstacle navigation
Wall Detection	Left and right distance sensors	Detect the robot's position relative to the walls
Pillar Detection	Pixy Camera	Detect red and green pillars for obstacle avoidance
Pillar Position Calculation	Pixy x and y coordinates	Determine the relative position of detected pillars
Heading Correction	Gyro Sensor	Maintain the required navigation heading
Parking Detection	Pink marker detection using Pixy Camera	Identify the parking area
Movement Control	Driving motor encoder	Control travelling distance
Steering Feedback	Steering motor encoder	Control the actual steering motor position
Direction Control	Floor colour detection	Determine CW or CCW navigation
Navigation Control	Loop -based navigation	Maintain a structured navigation sequence

Compared with Generation 1, the Generation 2 software introduced a more integrated obstacle navigation approach to address inconsistent responses to walls and coloured pillars. Gyro Sensor feedback, distance sensor readings, Pixy Camera detection, and encoder feedback were integrated into the control system, allowing the robot to adjust its steering according to detected field conditions while maintaining accurate heading and movement.

Rationale: The Generation 2 software was redesigned to combine multiple sensor inputs instead of relying mainly on a single navigation condition. Wall-distance feedback was added to improve the robot's response when moving close to the walls, while Pixy Camera detection was used to identify coloured pillars and the parking marker. Gyro feedback was maintained for heading control, and encoder feedback was used to control steering position and travelling distance. This integrated approach was intended to provide more controlled obstacle navigation and serve as a foundation for further optimisation in Generation 3.

2.6.2.3 Obstacle Detection, Avoidance and Navigation Algorithms

The Generation 2 Obstacle Challenge program uses several algorithms working together to control the robot during navigation. The algorithms were selected based on the problems identified during Generation 1 testing, particularly difficulties with wall clearance, pillar avoidance, heading control and parking. The main algorithms and their engineering justifications are described below.

2.6.2.3.1 Wall Detection and Avoidance Algorithm

The wall avoidance algorithm uses the left and right distance sensors to monitor the robot's distance from the surrounding walls. When the detected wall distance is below **120 mm**, a steering correction is applied.

```
If leftwall < 120 And wallCheck=1 And Math.Abs(@relativeHeading)<45 Then

  Speaker.Tone(100,220,50)
  leftwallturn = -74

Else

  leftwallturn = 0

EndIf

'-----------------------------------------------------
' Right Wall Avoidance
'-----------------------------------------------------

If rightwall < 120 And wallCheck=1 And Math.Abs(@relativeHeading)<45 Then

  Speaker.Tone(100,660,50)
  rightwallturn = 74

Else

  rightwallturn = 0

EndIf
```

Rasionale: The wall avoidance algorithm was introduced to reduce the risk of the robot moving too close to the wall during navigation. Distance-based detection allows the robot to respond to the actual position of the wall rather than relying only on the Gyro Sensor. This provides an additional input for steering correction when navigating through narrow sections of the field.

2.6.2.3.2 Camera-Based Pillar Detection and Position Estimation

The Pixy Camera is used to detect red and green pillars. The program obtains the x and y coordinates of the detected pillar and calculates its relative distance from a reference point.

```
'Calculate pillar distance
greenDistance = Math.SquareRoot(Math.Power(greenx-130,2)+Math.Power(greeny-255,2))
redDistance = Math.SquareRoot(Math.Power(redx-130,2)+Math.Power(redy-255,2))
```

The calculated distance is then used to determine the required pillar avoidance response.

Rasionale: The Pixy Camera was selected for pillar detection because the pillars can be distinguished by their colours. Using the detected pixel coordinates allows the program to estimate the relative position of the pillar and generate a steering response before the robot reaches it. This provides the robot with visual information for obstacle avoidance.

2.6.2.3.3 Position-Based Pillar Avoidance and Steering Correction

When a red or green pillar is detected within the defined detection range, the program calculates a steering correction based on the pillar's position in the camera image. The x-coordinate indicates the horizontal position of the pillar, while the y-coordinate is used together with the x-coordinate for closer pillars to adjust the steering response. Then, the robot uses the pillar's detected colour and position to determine the appropriate steering response and move away from the obstacle while maintaining the intended navigation path.

```
If greenDistance < followdistance Or redDistance < followdistance Then

  If greenDistance < pillardistance Then

    pillarturn=((avoidGreenpixel-greenx)*greeny/100)*0.74
    lastPillar=-1

  ElseIf redDistance < pillardistance Then

    pillarturn=((avoidRedpixel-redx)*redy/100)*0.74
    lastPillar=1
```

Rasionale: A position-based steering correction was used so that the robot could adjust its path according to the location of the detected pillar. This approach allows the steering response to change depending on where the pillar appears in the camera's field of view, rather than using

one fixed steering angle for every obstacle. This provides a more flexible response when pillars are detected at different positions and distances. Besides, a dedicated pillar avoidance algorithm was used because red and green pillars represent obstacles that require the robot to change its path during navigation. Instead of relying only on gyro correction, the robot uses visual information from the Pixy Camera to identify the pillar and determine its relative position. This allows the robot to respond specifically to the detected obstacle and reduce the risk of collision.

2.6.2.3.4 Parking Lot Detection and Parking Control

The Pixy Camera is also used to detect the pink parking marker. The horizontal position of the detected pink marker (pink) is used together with the travelling direction to determine the parking position.

```
Sub CheckParking

  While loopCount<5

    If cw=1 Then

      If pinkx>0 And pinkx<150 Then
        parking=loopCount
        Speaker.Tone(100,800,1)
      EndIf

    Else

      If pinkx>180 Then
        Speaker.Tone(100,800,1)
        parking=loopCount
      EndIf
```

Rasionale: The pink marker detection was implemented to provide a visual reference for identifying the parking area. Using the camera position of the marker allows the robot to determine when it has reached the parking section without relying only on a fixed travelling distance.

2.6.2.3.5 Integrated Steering Control Algorithm

The different steering corrections are combined to produce the final steering command:

```
turn=leftwallturn+rightwallturn+pillarturn+pinkturn+gyroturn
```

The final command is then used to control the steering motor.

Rasionale: The steering corrections were integrated into a single control output so that the robot could respond to different field conditions during the same navigation sequence. Wall, pillar, parking-marker and gyro information can therefore contribute to the steering decision without requiring separate control programs.

2.6.2.3.6 Encoder-Based Movement Control

The driving motor encoder is used to measure the distance travelled by the robot through the waitDegrees() function.

```
Function waitDegrees(in number degrees)

  dDegrees = Motor.GetCount("D")

  While Math.Abs(Motor.GetCount("D") - dDegrees) < degrees
    Steering()
  EndWhile

EndFunction
```

Rasionale: Encoder-based movement control was used to provide a more repeatable travelling distance. The steering function is continuously called while the robot is moving, allowing steering corrections to be applied without stopping the driving motor.

2.6.2.3.7 Floor-Colour Sensor-Based Navigation Direction Detection

At the beginning of the navigation sequence, the program uses the color sensor reading to determine whether the robot should travel clockwise or counter-clockwise.

```
If cw = 1 Then
  While Sensor.ReadRawValue(4,0) <> 4 And Sensor.ReadRawValue(4,0) <> 5
    Steering()
  EndWhile
Else
  While Sensor.ReadRawValue(4,0) <> 2
    Steering()
  EndWhile
EndIf
```

Rasionale: Automatic direction detection was used to allow the program to adapt to the detected starting condition. This reduces the need for a fixed navigation direction and allows the same control logic to support both clockwise and counter-clockwise movement.

2.6.2.4 Obstacle Challenge Performance Evaluation

The Generation 2 Obstacle Challenge program was tested to evaluate the performance of the integrated obstacle detection and avoidance system. The testing focused on wall avoidance, pillar detection, steering response, parking detection and overall navigation performance. The results were used to identify remaining limitations and determine the improvements required for Generation 3.

2.6.2.4.1 Obstacle Navigation Testing and Results Analysis

No.	Parking Exit	Without Parking Exit	Pillar Avoidance	Complete Obstacle 3 Round	Parking
1	Fail	-	-	-	-
2	Fail	-	-	-	-
3	Fail	-	-	-	-
4	Fail	-	-	-	-
5	Fail	-	-	-	-
6	-	Success	Success	Success	Partially
7	-	Success	Fail	Fail	Partially
8	-	Success	Fail	Fail	Partially
9	-	Success	Success	Success	Partially
10	-	Success	Success	Success	Partially

Overall Testing Analysis

The Generation 2 testing showed that the robot's main navigation system was functional when the parking exit sequence was excluded. However, the parking exit could not be completed successfully in the five dedicated trials. Pillar avoidance achieved a 60% success rate, indicating that the detection and steering system worked but remained inconsistent. The robot also achieved a 60% success rate when completing three obstacle rounds. Although the robot could reach the parking stage, the final parking performance remained only partially successful. These results identified parking exit, pillar avoidance reliability and parking accuracy as the main areas requiring further development in Generation 3.

2.7 Edge-Case Handling and Control Conflict Management

Edge Case	Potential Problem	Program Response
Multiplexer No Response	A sensor connected through the multiplexer may suddenly stop providing a valid value. Continuing to drive without valid sensor feedback may cause incorrect navigation or obstacle avoidance.	The program checks for a valid sensor response. If no valid value is received, the driving speed is set to **0**, causing the robot to stop temporarily. The robot resumes movement when a valid sensor response is received.
Multiple Steering Corrections Active	Wall, pillar or pink-marker detection may occur at the same time. Combining several corrections may produce an excessive or conflicting steering response.	Wall, pillar and pink-marker corrections are given priority. Gyro correction is disabled when any of these specific steering corrections is active.

2.7.1 Multiplexer Communication Failure and Safe-Stop Response

Program Response to Multiplexer Sensor No Response

```
If leftwallturn=0 And rightwallturn=0 And pillarturn=0 And pinkturn=0 Then
  gyroturn = @relativeHeading * correction * 2.2
Else
  gyroturn=0
EndIf
```

2.7.2 Steering Correction Prioritisation and Conflict Resolution

Multiple Steering Corrections Handling

```
'Driving condition
If leftwall>0 And rightwall>0 And Sensor.ReadRawValue(4,0)>0 Then
  Motor.StartPower("D",speed)
Else
  Motor.StartPower("D",0)
EndIf
```

2.8 Generation 2 Limitations and Engineering Decisions

Based on the Generation 2 development and testing, several issues were identified in the mechanical, sensor and software systems. These issues were analysed based on the observed robot behaviour and testing results. The main issues identified were as follows.

Evidence-Based Issue Analysis and Design Decisions

Issue Identified	Evidence from Generation 2 Testing	Engineering Decision	Engineering Reasoning
Steering gear reduction increased programming complexity	The 12T → 20T gear reduction provided a more controlled steering response and lower measured angle error, but required additional motor-to-wheel angle calculations.	Use a simpler steering transmission for further development.	Although gear reduction improved steering control, the additional angle conversion and calibration increased software complexity. A simpler steering relationship allows the steering angle to be calculated and adjusted more directly, reducing programming complexity while maintaining controllable steering behaviour.
Ultrasonic sensor orientation depended on mounting method	Both horizontal and vertical configurations generally operated without major detection problems. However, performance and practicality depended on how the sensors were locked to the robot body.	Select sensor orientation based on mounting stability, available space and interaction with the chassis.	The testing showed that orientation alone was not the main factor affecting performance. The sensor mounting method influenced stability, space usage and the relationship between the sensor and robot body. Therefore, sensor placement must be considered together with the mechanical design rather than independently.
Robot had	The robot failed all 5	Redesign the	The comparison between the

difficulty exiting the parking area	dedicated parking-exit trials (0/5). When the parking exit was excluded, navigation achieved 5/5 successful trials (100%).	front section to provide more effective clearance during parking exit while keeping the main robot body within the required dimensions.	two tests isolated the parking exit as a major source of failure. Increasing the usable front clearance could improve the initial manoeuvre without requiring the entire robot body to become larger. This decision balances manoeuvrability with dimensional constraints.
Pillar avoidance was inconsistent	Pillar avoidance achieved 3/5 successful trials (60%). The robot could detect and avoid pillars in some runs but failed in others.	Further optimize camera placement and pillar steering parameters.	The testing demonstrated that the Pixy Camera and position-based steering algorithm were functional but not sufficiently consistent. Improving sensor placement and tuning the steering response could provide earlier and more reliable obstacle reactions.
Complete three-round navigation was not fully reliable	The robot completed three obstacle rounds successfully in 3/5 trials (60%) when the parking exit was excluded.	Improve the interaction and tuning of navigation, sensor feedback and obstacle avoidance functions.	The result showed that individual functions could operate, but reliability decreased when several control functions operated together over a longer sequence. This indicated a system-integration issue rather than a single missing function.
Gyro reading occasionally stopped	During testing, the Gyro Sensor occasionally produced a reading that did not	Add software handling to detect abnormal or unchanged	The Gyro Sensor provides heading feedback for steering correction. If the reading becomes stuck, the software

updating	update normally. This could cause the calculated heading error to remain unchanged even when the robot's actual orientation changed.	Gyro readings and adjust the control response accordingly.	may use outdated heading information and generate an incorrect steering command. Therefore, the program was adjusted to handle abnormal Gyro behaviour and reduce the effect of a temporary sensor reading failure on navigation.
Parking performance was only partially successful	The robot reached the parking stage in the tested runs, but the final parking position was only partially successful.	Further calibrate parking distance and steering parameters.	The result showed that the parking sequence was functional but the final position was not sufficiently consistent. Further calibration was therefore required to improve repeatability without changing the entire navigation system.

Engineering Decision Summary and Implications for Generation 3: The Generation 2 development demonstrated that improving one subsystem could introduce new trade-offs in another subsystem. The geared steering system improved steering control but increased programming and calibration complexity. Sensor testing showed that ultrasonic orientation was generally workable in both configurations, but the mounting method and available space had a greater influence on practical implementation. Obstacle Challenge testing identified the parking exit as a major failure point, while pillar avoidance and complete navigation remained inconsistent. These findings were not treated as isolated problems. The team considered the interaction between the mechanical structure, sensor placement, software control and field constraints when determining the next design changes. For Generation 3, the identified issues were therefore used as engineering inputs for further mechanical redesign, sensor configuration changes and software optimisation.

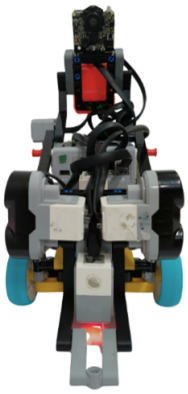
3.0 Generation 3: Design Optimisation and Control System Development

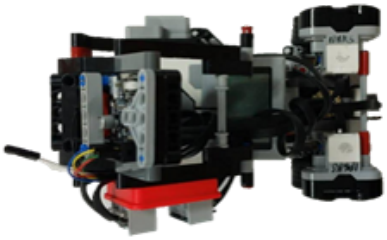
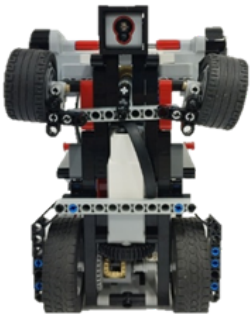
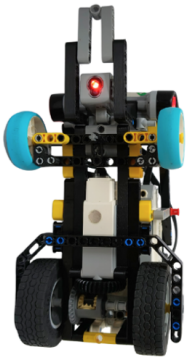
3.1 Overview

Based on the issues identified during Generation 2 testing, Generation 3 focused on improving the robot's mechanical manoeuvrability, sensor configuration and software reliability. The modifications were developed directly from the testing results and engineering decisions from Generation 2. The main objectives of Generation 3 were to improve parking exit performance, optimize sensor placement, simplify steering control and improve the reliability of sensor-based navigation.

3.2 Mechanical and Sensor Configuration Redesign

The mechanical redesign from Generation 2 to Generation 3 is shown in the table below.

Gen 2		Gen 3
Front View		 Front View
Back View		Back View

Left View	→	Left View
Right View	→	Right View
 Top View	→	Top View
 Bottom View	→	 Bottom View

Based on the Generation 2 testing, Generation 3 introduced several mechanical modifications to improve parking manoeuvrability, steering configuration and sensor integration. The front body was extended, the steering system returned to direct power steering, and the ultrasonic sensors and camera mounts were repositioned to accommodate the redesigned structure.

Mechanical/ Sensor Aspect	Generation 2	Generation 3	Engineering Reason
Front Body Structure	Original front geometry with limited parking manoeuvrability	Extended front chassis structure	Increase the available manoeuvring envelope and improve chassis geometry for dynamic parking operations.
Steering Mechanism	Geared-down steering with a reduction gear train	Direct-drive steering	Eliminate gear-ratio conversion in steering control, simplify motor-to-wheel angle mapping, and reduce control-system complexity.
Ultrasonic Sensor Orientation	Horizontal mounting	Vertical mounting	Adapt the sensor installation to the revised steering motor configuration and improve packaging within the front assembly.
Camera Position	Higher mounting position	Lower mounting position	Adjust the camera's viewing geometry to increase the intended field-of-view coverage and improve visual observation of the surrounding course.
Front Steering Wheels	Larger rim and tyre assembly	Smaller rim and tyre assembly using the Medium Azure Hard Tyre	Reduce wheel diameter for finer steering corrections. The harder rubber compound is intended to minimise bouncing and support more consistent straight-line tracking.

3.2.1 Front Chassis Extension for Parking Manoeuvrability

In Generation 3, the front chassis structure was extended to provide a more suitable geometry for dynamic parking manoeuvres. The previous Generation 2 configuration had limited maneuvering space during parking exit operations. By modifying the front structure, the team aimed to improve the robot's movement flexibility and provide a more suitable chassis layout for parking-related steering actions. This modification was intended to support smoother manoeuvring while maintaining the overall mechanical integration of the robot.

3.2.2 Steering Mechanism and Motor Configuration

Generation 3 replaced the geared-down steering mechanism used in Generation 2 with a direct-drive steering configuration. The geared system required the controller to account for the gear reduction ratio when translating motor rotation into steering angle. Direct drive simplifies this relationship by reducing the need for gear-ratio conversion calculations, making the steering control architecture more straightforward. The steering motor was also repositioned vertically to accommodate the revised mechanical layout and improve integration with the surrounding components.

3.2.3 Ultrasonic Sensor Orientation and Mounting Configuration

In Generation 2, the ultrasonic sensors were installed in a horizontal orientation. In Generation 3, the sensors were repositioned vertically to accommodate the revised steering motor configuration and front assembly layout. This change was made to improve component arrangement and reduce spatial conflicts around the steering mechanism. The revised mounting configuration was intended to maintain sensor integration while making more efficient use of the available space.

3.2.4 Camera Position and Field-of-View Optimisation

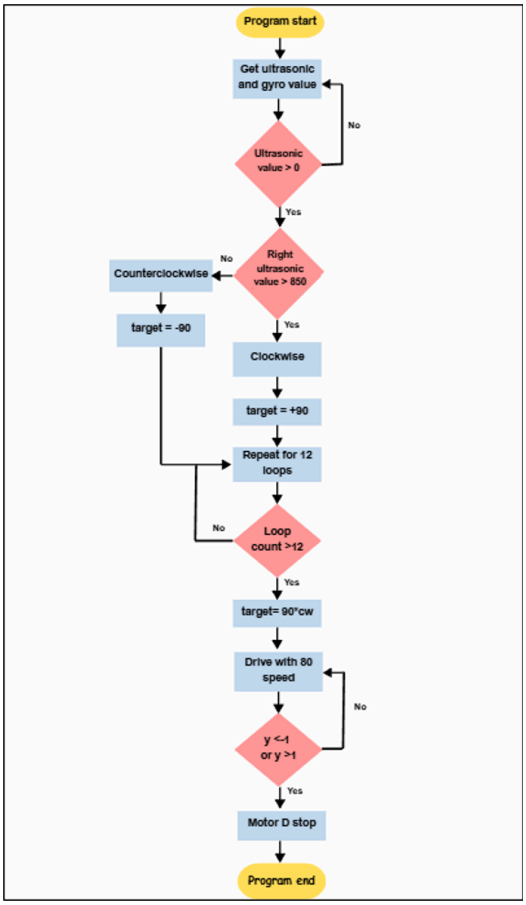
The camera mounting position was lowered in Generation 3 to increase the intended field-of-view coverage. Compared with the higher mounting position in Generation 2, the revised position was designed to provide a more suitable viewing angle for observing the competition environment. This adjustment was intended to improve the camera's visual coverage of relevant course features. Its actual effectiveness should be evaluated through camera detection tests under consistent operating conditions.

3.2.5 Front Steering Wheels and Tyre Selection

In Generation 3, the front steering axle was redesigned with a smaller rim and tyre assembly. The team selected the Medium Azure Hard Tyre from the SPIKE Essential set because its hard rubber compound was expected to minimise bouncing and support more reliable straight-line tracking compared with softer tyres. The smaller tyre diameter was also selected to enable finer steering corrections. A smaller wheel travels a shorter distance for the same angular rotation, allowing more precise adjustments in the robot's steering response. Combined with the harder rubber compound, this configuration was intended to improve directional stability and steering precision during course navigation.

3.3 Software Architecture and Control System Development

3.3.1 Open Challenge Navigation Control



The Generation 3 Open Challenge program was designed around a sequential control strategy that combines ultrasonic sensing, gyroscope feedback and a predefined movement routine. This architecture separates the initial direction-selection process from the repeated movement

sequence and final stopping stage, making the robot's behaviour more structured and predictable.

3.3.1.1 Sensor-Based Navigation Direction Selection

The program uses the right ultrasonic sensor and a threshold of 850 to determine the initial driving direction. This allows the robot to select its movement configuration based on the detected environment rather than relying on a manually fixed direction. The threshold acts as a decision boundary between the two movement strategies. However, the reliability of this decision depends on sensor positioning, measurement consistency and the suitability of the threshold for the actual course geometry.

3.3.1.2 Repeated Navigation Sequence

The 12-loop sequence provides a predefined movement pattern, reducing the need for continuous high-level route decisions during this stage. A fixed sequence can make the program easier to reproduce and debug because the number of repetitions is controlled. The trade-off is reduced adaptability: if the robot's position or heading deviates significantly during the routine, a fixed loop count alone does not guarantee that it will remain aligned with the intended path.

3.3.1.3 Gyroscope-Based Heading Control

The gyroscope is used to monitor the robot's angular state and support the final movement stage. The target heading is adjusted according to the selected direction using 90◦×cw90^\circ \times cw90◦×cw, allowing the same program structure to represent clockwise and counterclockwise movement. This reduces the need to maintain two separate movement routines. The final gyro condition provides a sensor-based stopping trigger rather than relying only on a fixed delay or estimated travel time.

3.3.2 Programming Algorithms and Control Implementation

3.3.2.1 Sensor Initialisation and Multiplexer Configuration

```
folder "prjs" "WRO2025"
Speaker.Tone(100,440,50)

Function setMultiplexerMode(in number port, in number channel, in number mode)
  address = 80 + 1 * (channel - 1)
  Sensor.WriteI2CRegister(port, address, 82, mode)
EndFunction

Function getMultiplexerValues(in number port, in number channel, out number values)
  address = 80 + 1 * (channel - 1)
  readData = Sensor.ReadI2CRegisters(port, address, 84, 2)
  values = readData[1] * 256 + readData[0]
EndFunction

Sensor.SetMode(2,0)
Sensor.SetMode(4,2)
setMultiplexerMode(3,2,0)
setMultiplexerMode(3,3,0)

Sub ReadSensor
  While "True"
    gyro = Sensor.ReadRawValue(2,0)
    relativeHeading = gyro-target
    getMultiplexerValues(3,2,leftWall)
    getMultiplexerValues(3,3,rightWall)
    LCD.Clear()
    LCD.Text(1,0,40,2,leftWall)
    LCD.Text(1,0,60,2,rightWall)
  EndWhile
EndSub
Thread.Run = ReadSensor
```

The program begins by configuring the sensors and multiplexer before movement starts. The setMultiplexerMode() function configures the multiplexer to communicate with both ultrasonic sensors through the same EV3 port, while getMultiplexerValues() retrieves the left and right distance measurements. The ReadSensor process continuously updates the robot's heading and wall-distance measurements. Separating sensor acquisition from the main movement sequence allows the control functions to use updated readings during navigation.

Engineering justification: The multiplexer configuration reduces the need for separate EV3 input ports for the two ultrasonic sensors, supporting a more compact sensor interface. Continuous sensor updates are important because wall-following decisions depend on the robot's current position relative to the surrounding boundaries. However, the reliability of the control system still depends on valid sensor readings and appropriate handling of missing measurements.

3.3.2.2 Robot Position Estimation and Coordinate Tracking

```
x=0
y=0
Sub GetXY
  While "True"
    x = x + (Motor.GetSpeed("D") * Math.Round(Math.Sin(Math.GetRadians(gyro))*10) * 0.1) * 0.001
    y = y + (Motor.GetSpeed("D") * Math.Round(Math.Cos(Math.GetRadians(gyro))*10) * 0.1) * 0.001
    If x>20 Then
      x = 20
    EndIf
    If x<-20 Then
      x=-20
    EndIf
    If y>20 Then
      y=20
    EndIf
    If y<-20 Then
      y=-20
    EndIf
    LCD.Clear()
    LCD.Text(1,0,0,2,x)
    LCD.Text(1,0,20,2,y)
  EndWhile
EndSub
```

The program estimates the robot's position along the X and Y axes by combining motor movement and gyroscope heading. The Math.Sin() and Math.Cos() functions resolve the movement into horizontal and vertical components according to the robot's orientation. The estimated coordinates are updated continuously and displayed on the EV3 LCD. The program also constrains both coordinate values to a range between −20 and 20.

Engineering justification: Coordinate tracking provides an additional representation of the robot's movement beyond its instantaneous heading. While the gyroscope indicates angular orientation, combining heading with motor movement allows the program to estimate how the robot has translated across the course. This is useful for the final positioning stage, where the robot uses its estimated Y-coordinate as a stopping reference.

3.3.2.3 Steering Reset and Reference Calibration

```
Sub ResetSteering
  Motor.StartPower("A",-100)
  Program.Delay(300)
  While Motor.GetSpeed("A")<>0
  EndWhile
  Program.Delay(100)
  Motor.ResetCount("A")
EndSub
```

Before driving, the Reset Steering function establishes a reference position for the steering motor. The medium motor connected to Port A turns fully to the left for 300 milliseconds, then waits for the motor to stop. Its rotation value is subsequently reset to zero.

Engineering justification: The steering reset establishes a consistent motor reference before closed-loop steering begins. Without a known starting reference, the same motor command could produce different wheel angles between runs, affecting the relationship between the controller's output and the robot's actual steering direction.

3.3.2.4 PID-Based Steering Control

```
intergal = 0
lastError = 0
Function PID_Steering(in number value1, in number value2, in number kp, in number ki, in number kd)
  error = value1 - value2
  p = error * kp

  @intergal = @intergal + error
  If @intergal>5 Then
    @intergal = 5
  ElseIf @intergal<-5 Then
    @intergal = -5
  ElseIf error = 0 Then
    @intergal = 0
  EndIf

  i = @intergal * ki

  d = (error-@lastError) * kd
  @lastError = error

  correction = p + i + d
  Motor.StartPower("A",correction)
EndFunction
```

The PID Steering function adjusts the steering motor according to the difference between the desired steering condition and the measured condition. The controller combines three components: proportional, integral and derivative control.

PID Component	Engineering Function
Proportional (P)	Responds to the current error, producing a correction related to the size of the deviation.
Integral (I)	Accumulates error over time to address persistent deviations that proportional control alone may not fully correct.
Derivative (D)	Responds to the rate of error change, helping moderate rapid corrections and reduce oscillatory behaviour.

Engineering justification: A PID controller was newly integrated into Generation 3 to improve closed-loop steering accuracy and reduce heading deviation during navigation. Unlike fixed steering commands, the controller continuously calculates the steering correction from the measured error, allowing the robot to respond dynamically to changes in its heading. The proportional term determines the immediate correction, the integral term compensates for accumulated error, and the derivative term responds to the rate of error change to regulate steering response. This integration introduces a feedback-based control mechanism that accounts for deviations during operation rather than relying solely on predefined steering inputs. The main engineering trade-off is the increased sensitivity to controller tuning: the PID gains must be calibrated to balance correction speed, overshoot and oscillation while maintaining stable steering performance.

3.3.2.5 Drive Control and Wall Following

```
trackWall=0
Function Drive(in number speed)
  If @leftWall = 0 Or @rightWall = 0 Then
    Motor.StartPower("D",0)
  Else
    Motor.StartPower("D",speed)
  EndIf

  If @trackWall = 1 Then
    If @cw = 1 And Math.Abs(@relativeHeading)<40 Then
      wallTurn = (170 - @rightWall) * 0.1 '-------------distance with wall-------------'
    ElseIf @cw = -1 And Math.Abs(@relativeHeading)<40 Then
      wallTurn = (@leftWall - 170) * 0.1 '200
    EndIf

    If wallTurn>40 Then
      wallTurn = 39
    ElseIf wallTurn<-40 Then
      wallTurn = -39
    EndIf
  EndIf

  turn = 43 + @relativeHeading + wallTurn '-------------reset steering-------------'
  PID_Steering(turn,Motor.GetCount("A"),6,0.1,5) 'steering sensitivity
EndFunction
```

Function Drive controls how the robot drives and follows a wall. First, it checks whether the ultrasonic sensors on both sides are working. If either sensor is not detecting a wall (value = 0), the robot stops moving. This acts as a safety feature, ensuring the robot doesn't move blindly when there is no sensor data. If both sensors are working, the robot moves forward at the given speed. Next, when wall tracking (trackWall) is turned on, the robot adjusts its direction to stay at the correct distance from the wall. If the robot is moving clockwise, it uses the right sensor; if

moving counterclockwise, it uses the left sensor. It calculates how much the robot needs to turn to maintain the proper distance and making sure it's not too close or too far. If the error is more than or less than 40 based on the robot's distance and the inner wall, it will perform a correction to the steering. Finally, a PID steering control ensures smooth and precise turns based on this calculation, keeping the robot aligned while following the wall safely.

Engineering Justification: The wall-following system uses direction-dependent sensor selection to measure the relevant wall and calculate distance error. This error is processed by the PID controller to adjust steering and maintain the intended wall clearance. A zero-reading safety check prevents movement when distance feedback is unavailable, reducing control decisions based on invalid data.

3.3.1.6 Main Program and Navigation Sequence

```
target = 0
cw = 0
trackWall = 0
Time.Reset1()
ResetSteering()
DriveDegrees(100,200,0)
While @leftWall<2000 And @rightWall<2000
  Drive(80)
EndWhile

If @rightWall>850 Then
  cw = 1
Else
  cw = -1
EndIf
```

```
For loopCount= 1 To 10
  trackWall = 1
  target = target + 90 * cw '90.25
  DriveDegrees(80,1350,0)
  Speaker.Tone(100,400,50)
  If cw = 1 Then
    While @rightWall<800
      Drive(80) '<-----------------Speed
    EndWhile
  Else
    While @leftWall<800
      Drive(80) '<-----------------Speed
    EndWhile
  EndIf
  Speaker.Tone(100,800,50)
EndFor

'---------------------------------------Get in last section-------------------------'

target = target + 90 * cw
While y<-1 or y>1
  Drive(80)
EndWhile
Motor.Stop("D","True")
Program.Delay(2500)
```

The main program coordinates the initial movement, direction selection, wall-following activation, repeated navigation and final positioning.

1. **Initialisation:** Reset the timers and steering reference, then begin moving forward.
2. **Direction selection:** Use the ultrasonic readings to determine whether the robot follows a clockwise or counterclockwise route.
3. **Initial movement:** Enter the second section and travel a predefined motor distance before enabling wall tracking.
4. **Wall-following navigation:** Activate trackWall and drive at a controlled speed of 80 while following the selected wall.
5. **Repeated movement:** Repeat the programmed sequence for 10 loops, turning 90◦90^\circ90◦ in the selected direction during each loop.

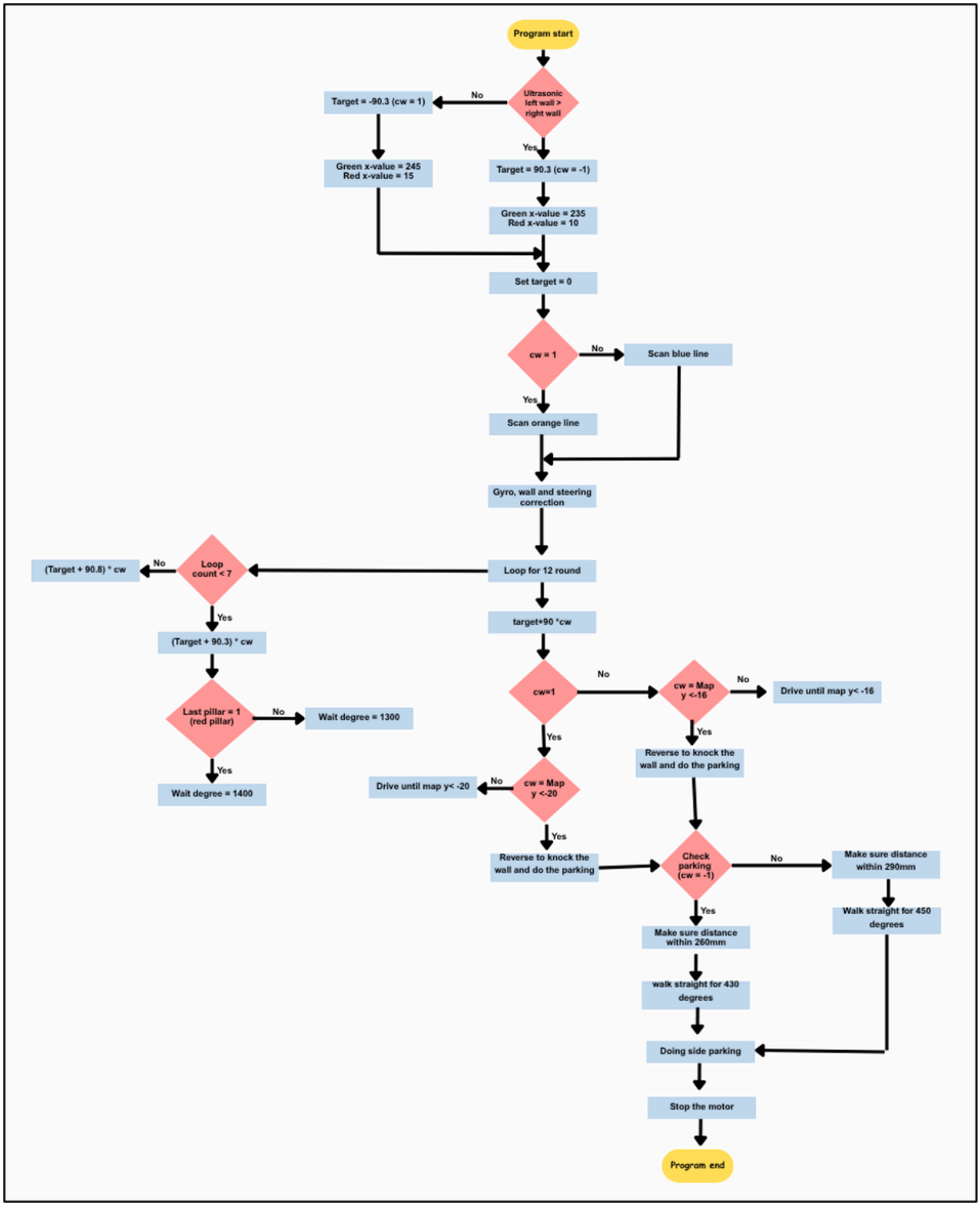
6. **Final positioning:** Update the target and continue driving while monitoring the estimated Y-coordinate.
7. **Stopping:** Stop the motors when the Y-coordinate is approximately within the range −1.

Engineering justification: The initial movement before activating wall tracking acts as a transition between the starting condition and closed-loop navigation. Immediately enabling wall tracking when the robot is far from its intended wall distance could produce a large initial error and an abrupt steering response. The predefined movement segment allows the robot to advance before applying wall-based correction, reducing the likelihood of an unnecessarily sharp initial adjustment. During navigation, the repeated sequence provides a structured route, while wall tracking supplies feedback to correct deviations as the robot moves. This combination balances predictable route execution with local environmental correction. In the final section, the estimated Y-coordinate provides a position-based stopping reference rather than relying exclusively on a fixed travel duration. Its accuracy, however, depends on the quality of the coordinate estimate.

Open Challenge Summary

The Generation 3 Open Challenge program combines sensor feedback, coordinate tracking, wall following, and PID steering control to improve navigation consistency. Ultrasonic sensors provide wall-distance feedback while the gyro and motor rotation are used to estimate the robot's position and movement. During wall-following, the appropriate sensor is selected according to the travel direction, and the measured distance error is processed by the PID controller to continuously adjust steering output. This feedback-based approach allows the robot to correct deviations during movement rather than relying only on fixed steering values.

3.3.3Obstacle Challenge Software Architecture and Control Algorithms



The Generation 3 Obstacle Challenge algorithm was developed as a sensor-feedback navigation and control system. The program combines Pixy2 vision, ultrasonic sensing, gyro feedback, colour detection, position estimation and PID steering to control the robot through the obstacle course. Instead of using fixed movement commands alone, the program continuously converts sensor measurements into decisions and steering corrections. The overall control structure follows the sequence of sensing → state identification → error calculation → control output → motion → state transition, allowing the robot to maintain a predefined navigation strategy while responding to changes in its actual position and heading.

3.3.3.1 System Initialisation and Sensor Configuration

```
folder "prjs" "WRO2025"

Function setLampOn(in number port)
    Sensor.WriteI2CRegister(port,0,98,1)
EndFunction

Function setLampOff(in number port)
    Sensor.WriteI2CRegister(port,0,98,0)
EndFunction

Function getSignature(in number port, in number sig, out number x, out number y)
    address = 80 + sig
    values = Sensor.ReadI2CRegisters(port, 1, address, 3)
    x = values[1]
    y = values[2]
EndFunction

Function setMultiplexerMode(in number port, in number chanel, in number mode)
    address = 80 + 1 * (chanel - 1)
    Sensor.WriteI2CRegister(port,address,82,mode)
EndFunction

Function getMultiplexerValue(in number port, in number chanel, out number values)
    address = 80 + 1 * (chanel - 1)
    readData = Sensor.ReadI2CRegisters(port,address,84,2)
    values = readData[1] * 256 + readData[0]
EndFunction
```

```
Sub ReadSensor
  While "True"
    getSignature(1,1,greenx,greeny)
    getSignature(1,2,redx,redy)
    getSignature(1,3,pinkx,pinky)

    gyro = Sensor.ReadRawValue(2,0)
    relativeHeading = gyro - @target

    getMultiplexerValue(3,2,leftWall)
    getMultiplexerValue(3,3,rightWall)
    LCD.Clear()
    LCD.Text(1,0,40,2,leftWall)
    LCD.Text(1,0,60,2,rightWall)
  EndWhile
EndSub

Thread.Run = ReadSensor
```

```
Function GyroReset(in number port)
    'put the gyro into angle mode before trying to reset.
    Sensor.SetMode(port, 0)
    Sensor.ReadRawValue(port, 0)
    'write reset cmd
    gyroResetData[0] = 17
    Sensor.SendUARTData(port, 1, gyroResetData)
    'wait it apply
    Sensor.Wait(port)
EndFunction
```

The program first initializes the Pixy2 Camera, EV3 Gyro Sensor, EV3 Colour Sensor and ultrasonic sensors. The multiplexer is configured so that both ultrasonic sensors can be accessed through the same EV3 port, while the Pixy2 is configured to identify the green pillar, red pillar and magenta parking area using different signatures. The ReadSensor function continuously updates the pillar coordinates, gyro angle and wall-distance measurements during operation. Before movement begins, GyroReset establishes a zero-heading reference and ResetSteering establishes a known steering position. The purpose of this stage is to ensure that all sensors and actuators begin from a consistent reference. From a computer-control perspective, the sensor values are converted into variables that can later be used as decision inputs, while the engineering decision to reset the gyro and steering before each run reduces the effect of initial offsets and improves repeatability.

3.3.3.2 Robot Position Estimation and Coordinate Tracking

```
dCounted=0
Mapx=0
Mapy=0
Sub GetXY
  While "True"
    Mapy = Mapy + (Motor.GetSpeed("D") * Math.Round(Math.Cos(Math.GetRadians(gyro))*10)
* 0.1) * 0.001
    If cw=1 Then
      If Mapy>40 Then
        Mapy=40
      EndIf
      If Mapy<-20 Then
        Mapy=-20
      EndIf
    Else
      If Mapy>25 Then
        Mapy=25
      EndIf
      If Mapy<-40 Then
        Mapy=-40
      EndIf
    EndIf
    LCD.Clear()
    'LCD.Text(1,0,0,2,Mapx)
    LCD.Text(1,0,20,2,Mapy)
  EndWhile
EndSub

Thread.Run = GetXY
```

The program estimates the robot's Y-axis position using drive motor movement together with the heading measured by the gyro. The cosine of the gyro angle is used to determine the component of the robot's movement contributing to the Y-axis, and the calculated value is accumulated in MapY. This value is continuously updated and displayed on the EV3 LCD, while different operating limits are applied according to the selected navigation direction. The function of MapY is to provide a software-based positional reference for later state transitions. The logical process is to convert motor movement and heading information into an estimated coordinate rather than relying on motor rotation alone. From an engineering perspective, this provides additional positional information without requiring a dedicated positioning sensor, although the estimation remains affected by wheel slip and gyro error. Therefore, range limits are applied to prevent unrealistic positional values from influencing later decisions.

3.3.3.3 Steering Reset and Reference Calibration

```
Sub ResetSteering
    Motor.StartPower("A",-100)
    Program.Delay(300)
    While Motor.GetSpeed("A")<>0
      'wait until stalled
    EndWhile
    Motor.ResetCount("A")
EndSub
```

The ResetSteering function establishes a repeatable steering reference before the robot begins the Obstacle Challenge. The steering motor first rotates the steering mechanism towards its mechanical limit, allowing the program to identify a known reference position before resetting the motor position. From a programming perspective, this creates a fixed starting coordinate for the steering system, so subsequent steering commands can be calculated relative to the same reference. This is important because the robot's steering response depends on the relationship between the motor position and the actual wheel angle. The engineering decision to recalibrate the steering at the beginning of each run reduces the effect of mechanical positioning differences and provides a more consistent steering response throughout the challenge.

3.3.3.4 PID-Based Steering Control

```
intergal = 0
lastError = 0
Function PID(in number turn, in number kp, in number ki, in number kd)
  error = (44 + turn) - Motor.GetCount("A")
  p = error * kp

  @intergal = @intergal + error
  If @intergal>5 Then
    @intergal = 5
  ElseIf @intergal<-5 Then
    @intergal=-5
  ElseIf error=0 Then
    @intergal = 0
  EndIf
  i = @intergal*ki

  d = (error-@lastError)*kd

  result = p+i+d
  Motor.StartPower("A",result)
  @lastError = error
EndFunction
```

The PID Steering function converts the calculated steering error into a motor command for the steering motor. The controller consists of proportional, integral and derivative components, which respond respectively to the current error, accumulated error and rate of error change. These components are combined to generate the final steering output sent to Motor A. The main function of the PID controller is to continuously correct the robot when its actual heading deviates from the required steering target. From a computer-control perspective, the process forms a feedback loop in which the robot measures its deviation, calculates an error, processes the error through the PID controller and then changes the steering position before measuring the new error again. PID control was newly introduced in Generation 3 to replace a more fixed steering response and provide greater control over heading stability after pillar avoidance. The engineering trade-off is that the controller requires appropriate parameter tuning; excessive correction can cause oscillation, while insufficient correction can result in slow or incomplete recovery.

3.3.3.5 Integrated Drive and Obstacle-Tracking Control

The Drive function acts as the main motion-control block by combining TrackWall, TrackPillar and PID Steering to determine the required steering correction while Motor D provides forward movement. Instead of treating wall tracking and pillar avoidance as independent movements, the program combines their outputs into a steering decision before sending the final command to the steering motor. This creates a layered control structure in which the sensors provide environmental information, the tracking functions convert that information into steering error, and the PID controller converts the error into motor movement.

(i) Wall Tracking and Distance-Based Correction

```
If (@leftWall<150 Or @rightWall<150) And @trackWall=1
  If @leftWall<150 Then
    turn = -30
  Else
    turn = 30
  EndIf
Else
```

```
  If @leftWall < @rightWall Then
    followWall = (@leftWall - @wallDistance) * 0.2 'sensitivity
  Else
    followWall = (@rightWall - @wallDistance) * -0.2
  EndIf
Else
  followWall = 0
EndIf
```

The TrackWall function uses the left and right ultrasonic sensors to determine the robot's relationship with the surrounding walls. The program checks whether a wall is detected within the programmed 150 mm range and compares the two distance readings to determine which wall provides the relevant reference. If the left wall is closer, the program generates the corresponding steering correction toward the left-side reference; if the right wall is closer, the correction is applied in the opposite direction. When neither wall satisfies the tracking condition, no wall correction is generated and the robot continues moving forward. The computer logic therefore follows distance measurement → range check → wall selection → steering error. The engineering decision to apply wall correction only when a suitable wall is detected prevents the robot from making unnecessary steering movements in open areas. This reduces the possibility of over-correction and allows the robot to maintain a more stable trajectory.

(ii) Camera-Based Pillar Tracking and Object Classification

```
'Calculate pillar distance
greenDis = Math.SquareRoot(Math.Power(@greenx-130,2)+Math.Power(255-@greeny,2))
redDis = Math.SquareRoot(Math.Power(@redx-130,2)+Math.Power(255-@redy,2))
pinkDis = Math.SquareRoot(Math.Power(@pinkx-130,2)+Math.Power(255-@pinky,2))
```

```
'Calculate pillar distance
greenDis = Math.SquareRoot(Math.Power(@greenx-130,2)+Math.Power(255-@greeny,2))
redDis = Math.SquareRoot(Math.Power(@redx-130,2)+Math.Power(255-@redy,2))
pinkDis = Math.SquareRoot(Math.Power(@pinkx-130,2)+Math.Power(255-@pinky,2))

'Follow or Avoid Pillar
If (greenDis<155 Or redDis<150 Or pinkDis<150) And trackPillar=1 Then 'start showing LED color
  If greenDis<redDis And greenDis<pinkDis Then
    @lastPillar = 1
    EV3.SetLEDColor("GREEN","NORMAL")
    If @greeny>150 Then
      steeringTurn = (@greenAvoid-@greenx)*@greeny/100*0.6 'avoiding pillars
    Else
      steeringTurn = (@greenFollow-@greenx)*@greeny/100*0.5 'keep car in the middle
    EndIf
  ElseIf redDis<greenDis And redDis<pinkDis Then
    @lastPillar = 2
    EV3.SetLEDColor("RED","NORMAL")
    If @redy>160 Then
      steeringTurn = (@redAvoid-@redx)*@redy/100*0.6
    Else
      steeringTurn = (@redFollow-@redx)*@redy/100*0.5
    EndIf
  Else
    EV3.SetLEDColor("ORANGE","NORMAL")
    If @cw=1 Then
      steeringTurn = (0-@pinkx) * @pinky/100*0.55
    Else
      steeringTurn = (255-@pinkx) * @pinky/100*0.5
    EndIf
  EndIf
Else
```

The TrackPillar function uses the Pixy2 Camera to determine the position and relative distance of detected pillars. The program calculates the distance between the detected object position and the camera reference using the X and Y coordinates, applying the Pythagorean relationship to obtain the relative straight-line distance. The program then identifies whether the closest relevant object is a green pillar, red pillar or magenta parking area. When the green pillar

satisfies the programmed detection condition, @lastPillar is assigned 1 and the green LED is activated. When the red pillar satisfies its condition, @lastPillar is assigned 2 and the red LED is activated. The vertical position of the detected pillar is also used as a trigger for steering intervention, with the programmed detection conditions determining when the robot should begin responding to the obstacle. When the magenta parking area is detected, the orange LED is activated and the robot applies a direction-dependent steering response to avoid the parking area. The computer logic therefore converts camera coordinates → distance estimation → object classification → navigation state → steering response. The engineering decision to use both pillar colour and camera position allows the robot to determine not only what obstacle is present, but also when the obstacle is close enough to require a steering action.

```
EV3.SetLEDColor("OFF","NORMAL")  '---------pd value of ultrasonic sensor-------'
If @cw=1  And Math.Abs(@relativeHeading) < 35 Then
  followWall = (@leftWall - @wallDistance) * 0.2
ElseIf @cw=-1 And Math.Abs(@relativeHeading) < 35 Then
  followWall = (@rightWall - @wallDistance) * -0.2
Else
  followWall = 0
EndIf
```

When no pillar requires active tracking, the program returns to ultrasonic wall tracking. During clockwise movement, the programmed wall reference is on the left side, while during counterclockwise movement the reference is on the right side. If the robot's relative heading has already deviated significantly from the intended direction, the program avoids applying additional wall correction. This condition is used to prevent the wall-following controller from making an excessive correction while the robot is already recovering its heading. The engineering decision is therefore to prioritise controlled recovery over continuously applying multiple corrections that could cause unstable steering.

(iii) Steering Error Calculation and Control Integration

```
      turn = @relativeHeading + followWall

    EndIf
  EndIf

  PID(turn,3,0.1,2)
EndFunction
```

The final steering error is generated from the active pillar-tracking and wall-tracking conditions. These corrections are combined and passed to the PID steering controller, which determines

the required steering position of Motor A. At the same time, Motor D drives the robot forward at the programmed runSpeed. This creates a closed-loop control process in which environmental measurements are converted into an error, the error is processed by PID control, and the resulting steering command changes the robot's trajectory. The separation between steering correction and forward drive allows the robot to maintain continuous movement while dynamically adjusting its heading.

3.3.3.6 Supporting Movement Control Functions

The additional movement functions provide controlled distance and steering operations that support the main navigation algorithm. These functions separate basic motion control from the higher-level obstacle decision-making, making the overall program easier to control and repeat.

(i) Encoder-Based Movement Control (`WaitDegrees`)

```
Function waitDegrees(in number speed, in number degrees, in number stop)
  Motor.ResetCount("D")
  While Math.Abs(Motor.GetCount("D"))<degrees
    Drive(speed)
  EndWhile
  If stop=1 Then
    Motor.Stop("D","True")
    Program.Delay(50)
  EndIf
EndFunction
```

The WaitDegrees function moves the robot for a specified amount of drive-motor rotation using Motor D. The motor rotation is used as an encoder-based movement reference, allowing the program to perform a defined amount of movement before continuing to the next instruction. When stop = 1, the motor stops after reaching the target rotation and the program waits for approximately 50 ms before continuing. This short settling period prevents the next movement command from being executed immediately after the motor stops, providing a more controlled transition between movement stages. The engineering decision to use motor rotation as a movement reference provides repeatable motion without relying entirely on timing, although the actual travelled distance can still be affected by wheel slip and traction.

(ii) Combined Movement and Steering Control (`SteeringDrive`)

```
Function SteeringDrive(in number steering, in number speed, in number degrees, in number stop)
  Motor.ResetCount("D")
  While Math.Abs(Motor.GetCount("D"))<degrees
    If Sensor.ReadRawValue(4,0)=0 Or @leftWall=0 Or @rightWall=0 Then
      Motor.StartPower("D",0)
    Else
      Motor.StartPower("D",speed)
    EndIf

    PID(steering,3,0.1,2)

  EndWhile
  If stop=1 Then
    Motor.Stop("D","True")
    Program.Delay(50)
  EndIf
EndFunction
```

The SteeringDrive function combines forward movement with steering control. It allows the robot to travel a specified distance while maintaining the current steering position and applying an additional steering adjustment when required. This function is useful during both challenge sequences because movement and steering can be controlled within the same operation rather than treating them as completely independent commands. From a programming perspective, this creates a reusable movement block that can accept both distance and steering information, reducing repeated code and providing more consistent motion behaviour across different sections of the program.

3.3.3.7 Main Program and Obstacle Navigation Sequence

```
If leftWall>rightWall Then  '-----------ccw-------------'
  Speaker.Tone(100,1000,50)
  target=90.3
  cw=-1
```

The Main program coordinates the complete Obstacle Challenge sequence by combining the sensing, decision-making and movement functions. At the beginning of the run, the robot compares the left and right ultrasonic wall distances to determine its navigation direction. When leftWall > rightWall, the program selects counterclockwise navigation, while rightWall > leftWall selects clockwise navigation. The selected direction is stored as a program state and is subsequently used to determine the appropriate steering targets and obstacle-avoidance behaviour. This allows one control structure to support both directions while accounting for the different geometry encountered by the robot.

```
  'Pillar X value
  greenAvoid=225
  greenFollow=120
  redAvoid=10
  redFollow=80

Else
  Speaker.Tone(100,200,50)
  target=90
  cw=1

  'Pillar X value
  greenAvoid=235
  greenFollow=160
  redAvoid=10
  redFollow=100
```

The pillar X-position is then used to determine where the pillar appears within the Pixy2 camera's field of view. This information is used to adjust the robot's path when approaching or avoiding a pillar. Rather than treating the pillar as a simple colour-detection event, the program uses its camera position as a spatial reference. This allows the robot to determine whether the pillar is positioned appropriately relative to the robot and whether a steering correction is required.

```
target=0
trackWall = 1
trackPillar=1
mainSpeed=45 '<-----------------------------------Main speed

If cw=1 Then
  While Sensor.ReadRawValue(4,0)<>5
    Drive(mainSpeed*0.9)
  EndWhile
Else
  While Sensor.ReadRawValue(4,0)<>0 And Sensor.ReadRawValue(4,0)<>2 And
Sensor.ReadRawValue(4,0)<>7
    Drive(mainSpeed*0.9)
  EndWhile
EndIf
Speaker.Tone(100,600,100)

setLampOn(1)
```

After exiting the parking area, the robot drives forward at a speed of 45 while monitoring the colour sensor. The robot continues until either a blue or orange line is detected. Once the target line is identified, the robot produces a sound to confirm the detection before entering the main obstacle-navigation state. This creates a clear transition between the starting movement and obstacle-navigation algorithm based on an actual field reference.

```
For loopCount = 1 To 11
  @redH = 170
  @greenH = 170
  followWallLogic = 1
  If loopCount<7 Then
    target = target + 90 * cw
  Else
    target = target + 90.5 * cw
  EndIf
```

During the Obstacle Round, the robot continuously calculates steering corrections using TrackPillar and TrackWall, with the resulting error processed by the PID steering controller. The redH and greenH values represent the programmed pillar-distance conditions obtained from the camera view and are used to determine when pillar tracking should become active. The steering target is adjusted according to the selected clockwise or counterclockwise direction, since the required correction is not symmetrical between the two navigation paths. This steering process is repeated for 11 loops, allowing the robot to follow the programmed obstacle sequence while continuously correcting its trajectory from sensor feedback.

```
If cw=1 Then
  If lastPillar = 1 Then
    degrees = 1600 'cw green is last
  Else
    degrees = 1450 'cw red is last
  EndIf
Else
  If lastPillar = 2 Then
    degrees = 1450 'ccw red is last
  Else
    degrees = 1600 'ccw green is last
  EndIf
EndIf

waitDegrees(mainSpeed,degrees,0)
Speaker.Tone(100,1500,100)
```

The program also includes an additional turning condition for specific pillar-avoidance situations. When the robot passes a pillar on its left side, corresponding to a green pillar during a clockwise run or a red pillar during a counterclockwise run, an additional steering rotation is applied. The purpose is to ensure that the robot completes the avoidance manoeuvre with a sufficient heading change. Without this additional correction, the robot could finish the avoidance movement at an incorrect orientation and potentially detect the wrong coloured line, causing the subsequent navigation decision to become incorrect. This represents an engineering decision based on the interaction between obstacle avoidance and subsequent sensor detection, rather than treating each movement as an isolated action.

```
If cw=1 Then
  While Mapy<4
    Drive(mainSpeed)
  EndWhile
Else
  While Mapy<-16
    Drive(mainSpeed)
  EndWhile
EndIf
```

After the 11 obstacle loops are completed, the program uses MapY to determine when the robot has fully entered the final section. During clockwise navigation, the robot continues forward while MapY < 4, while during counterclockwise navigation it continues until MapY < -16. These conditions act as state-transition thresholds, ensuring that the robot does not begin the reverse sequence before reaching the required section of the course. The use of MapY provides a positional decision rather than relying only on a fixed time or motor duration.

```
If cw=1 Then
  While Mapy>-10
    Drive(-35)
  EndWhile
Else

  While Mapy>-25
    Drive(-35)
  EndWhile
EndIf

followWallLogic = 1
wallDistance = 300
followWallResponse = 0.5

Time.Reset7()
While Time.Get7()<3000
  Drive(-35)
EndWhile
```

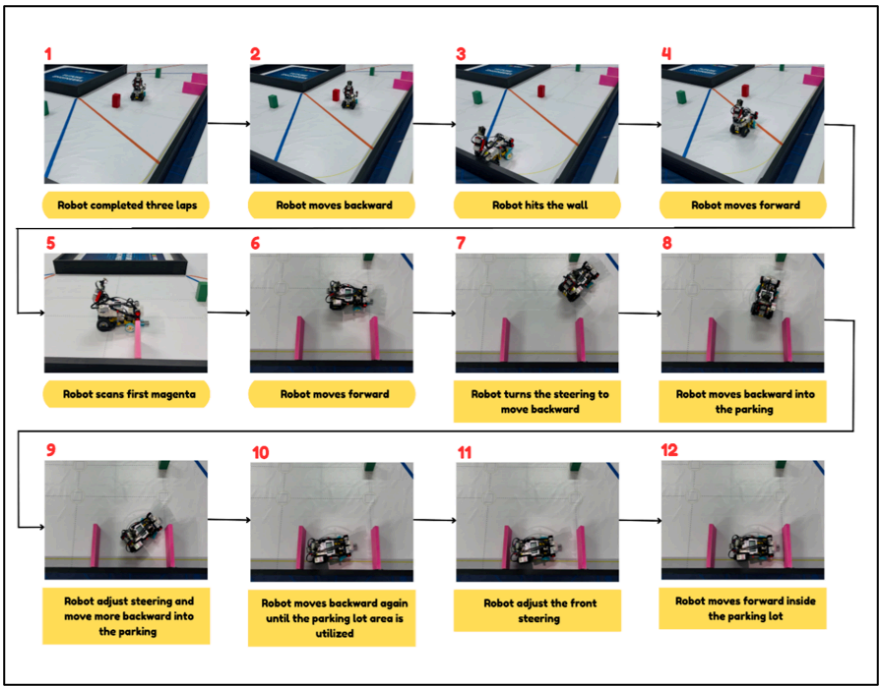
The robot then reverses toward the outer wall using direction-dependent MapY thresholds. During clockwise movement, it reverses until MapY < -10, while during counterclockwise movement it continues until MapY < -25. This movement is designed to bring the robot into a controlled relationship with the outer wall so that the wall can be used as a physical alignment reference. After reaching the required position, wall-following is enabled with a target distance of 300 mm and a response value of 0.5. The robot then continues reversing for approximately three seconds to establish the final position before starting the parking approach. The engineering decision to use the wall as a physical reference reduces the effect of accumulated positional error from the obstacle section.

```
If cw = -1 Then
  wallDistance = 260
  waitDegrees(mainSpeed,800,0)
  Speaker.Tone(50,250,500)
  While rightWall>130
    Drive(mainSpeed *0.7)
  EndWhile
  Speaker.Tone(50,200,50)
  followWallLogic=1
  waitDegrees(mainSpeed*0.7,430,1)
  Speaker.Tone(50,200,50)
  Program.Delay(200)
  SteeringDrive(60,40,235,1)
  SteeringDrive(-60,-40,180,1)
  SteeringDrive(60,-40,370,1)
  SteeringDrive(-6,30,25,1)
```

```
  wallDistance = 260
  waitDegrees(mainSpeed,400,1)
  Speaker.Tone(50,250,500)
  While leftWall>130
    Drive(mainSpeed*0.7)
  EndWhile
  Speaker.Tone(50,50,50)
  followWallLogic=1
  waitDegrees(mainSpeed*0.75,450,1)
  Speaker.Tone(50,200,50)
  Program.Delay(200)
  SteeringDrive(-60,40,215,1)
  SteeringDrive(60,-50,150,1)
  SteeringDrive(-60,-40,375,1)
  SteeringDrive(6,30,25,1)
EndIf
Brake(1)
Motor.Stop("D","True")
Speaker.Tone(100, 500, 100)
```

During the final side-parking sequence, the robot moves forward while maintaining approximately 260 mm from the parking area. The distance is continuously monitored and compared with the target value. When the measured distance deviates from the desired value, the steering is adjusted to bring the robot back toward the target distance. After passing the first magenta parking area, the robot reverses into the selected parking space and continues the maneuver until it reaches the required parallel orientation.

Sample for Procedure of Parking



3.3.3.8 Obstacle Challenge Control Architecture Summary

The Generation 3 Obstacle Challenge program was designed as a closed-loop, sensor-based navigation system rather than a sequence of fixed motor commands. The robot first converts sensor inputs from the Pixy2 Camera, ultrasonic sensors, colour sensor and gyro into meaningful states such as navigation direction, pillar position, wall position and MapY. These states are then processed through conditional logic to determine which control strategy should be active. During obstacle avoidance, TrackPillar provides obstacle-related information while TrackWall provides environmental positioning, and their steering errors are passed to the PID controller to generate the steering output for Motor A. Motor D maintains forward movement, creating a continuous feedback loop of sense → calculate error → decide → correct → sense again. After the 11 navigation loops, MapY is used as a state-transition condition to move from obstacle avoidance to wall alignment and finally to side parking. The main engineering decision was to combine predefined navigation states with real-time feedback control, allowing the robot to retain a predictable route while adapting its steering to actual sensor measurements. The introduction of PID in Generation 3 further improves this architecture by replacing purely fixed steering corrections with dynamic error-based control. Overall, the algorithm demonstrates a structured computer-thinking approach of input, processing, decision, control and feedback, with the main trade-offs being sensor reliability, position-estimation error, wheel traction and PID tuning.

3.3.4 Performance Testing and Evaluation

3.3.4.1 Open Challenge Testing and Reliability Analysis

Test Round	Time (s)	Success	Remark
1	24	Yes	-
2	23	Yes	-
3	7	No	Gyro Reading Error
4	25	Yes	-
5	25	Yes	-
6	30	No	Robot stuck against wall due to wiring interference
7	24	Yes	-
8	22	Yes	-
9	15	No	Gyro Reading Error
10	23	Yes	-
Total / Average	23.7 s*	7 / 10 Successful	70% Success Rate

Testing Result: The Generation 3 Open Challenge was tested over 10 rounds to evaluate the robot's reliability and consistency. The robot achieved 7 successful runs out of 10, giving a 70% success rate. The successful runs were completed within 22–25 seconds, with an average completion time of 23.7 seconds. Two failures were caused by gyro reading errors, which affected the robot's heading and resulted in incorrect navigation. Another failure occurred when the robot became stuck against the wall due to wiring interference. These failures showed that the robot's performance was affected not only by the control algorithm, but also by the physical arrangement of the sensors and wiring. The testing confirmed that the Generation 3 navigation system was functional under normal conditions, but improvements were needed in heading accuracy and cable management. These findings were used to guide the Generation 4 redesign, including repositioning the gyro to the front-centre of the robot and organising the multiplexer and wiring through the centre of the chassis.

3.3.4.2 Obstacle Challenge Testing and Performance Analysis

Test Round	Time(s)	Obstacle Situation after 3 laps	Parking	Mission Complete	Remark
1	95	Moved	Parallel Parking	Yes	Obstacle avoidance was initiated too late when the obstacle was detected at a greater distance
2	91	Moved	Partial Parking	Yes	Obstacle avoidance was initiated too late when the obstacle was detected at a greater distance
3	30	Not Moved	No	No	Gyro Reading Error
4	99	Moved	Partial Parking	Yes	Obstacle avoidance was initiated too late when the obstacle was detected at a greater distance
5	85	Moved	No	No	Robot wiring interfered with the parking lot
6	96	Not Moved	Parallel Parking	Yes	-
7	92	Not Moved	Parallel Parking	Yes	-
8	90	Moved	Parallel Parking	Yes	Obstacle avoidance was initiated too late when the obstacle was detected at a greater distance
9	97	Moved	Partial Parking	Yes	Obstacle avoidance was initiated too late when the obstacle was detected at a greater distance
10	6	Moved	No	No	Gyro Reading Error

Testing Result: The testing identified three main system-level issues. First, the camera mounting height was insufficient, which limited the forward detection range and caused the robot to detect obstacles later than required. This reduced the available distance for steering correction during obstacle avoidance. Second, gyro reading errors affected the accuracy of the robot's heading information, resulting in incorrect navigation behaviour in some runs. Third, the existing cable arrangement created physical interference during parking, showing that wiring layout could directly affect robot mobility. These findings were used to determine the Generation 4 design changes. The camera will be mounted higher using a custom-designed camera holder to increase the detection range while maintaining the required field of view. The gyro will be relocated to the front-centre of the robot above the steering mechanism to provide a more consistent heading reference. The wiring and multiplexer will also be reorganised through the central section of the chassis to minimise interference with the robot's movement.

3.3.5 Generation 3 Limitations and Engineering Decisions

Testing of Generation 3 identified several limitations in the mechanical structure, heading measurement, camera positioning, wiring arrangement and wheel traction. These issues did not necessarily prevent the robot from completing the challenge, but they affected the consistency, manoeuvrability and reliability of the system. The findings were used to define the design priorities for Generation 4.

Generation 3 Testing Issue	Generation 4 Engineering Decision	Expected Improvement
Steering wheel had limited grip	Same-size high-grip steering wheel	Better traction and steering consistency
Gyro heading error	Move gyro to front-centre above steering	More consistent heading reference
LEGO chassis limited design freedom	Integrated 3D-printed chassis	More dynamic and optimised structure
Camera was wide but detection distance was limited	Higher custom camera holder	Maintain wide FOV while improving viewing distance
Wiring could interfere with movement	Centralise multiplexer and wiring	Cleaner layout and reduced cable interference

3.3.5.1 Steering Wheel Traction and Directional Stability

The steering wheels used in Generation 3 had limited grip with the competition surface. Although the wheel size was suitable for the steering mechanism, insufficient traction could cause the wheels to slip slightly during steering and movement. This introduced a difference between the commanded steering position and the actual movement of the robot. For Generation 4, the steering wheels will be changed to a higher-grip wheel with the same overall size. Maintaining the same size allows the existing steering geometry to be retained while increasing friction between the wheel and the surface. This is intended to improve steering consistency without significantly changing the mechanical dimensions of the robot.

3.3.5.2 Gyroscope Heading Accuracy and Sensor Placement

During testing, the gyro reading showed heading errors that affected the robot’s direction estimation. Since the navigation algorithm uses heading information for steering correction and position estimation, an inaccurate heading can propagate into later navigation decisions. To address this issue, the Generation 4 gyro will be repositioned to the front-centre of the robot, directly above the steering mechanism. Placing the sensor closer to the front centreline provides a more suitable reference point for the robot's heading and reduces the effect of the previous mounting position on heading measurement. The design decision is therefore focused on obtaining a more consistent heading reference for the navigation algorithm.

3.3.5.3 LEGO Chassis Constraints and Structural Redesign

The Generation 3 chassis was constructed mainly from LEGO components. While the modular structure allowed rapid modification, the available LEGO geometry limited the freedom to optimize the chassis shape, component placement and overall mechanical layout. For Generation 4, the chassis will therefore move towards a custom 3D-printed integrated design. Most of the structural parts will be redesigned as 3D-printed components. This allows the dimensions, mounting positions and shape of the robot to be designed specifically around the required mechanisms and sensors. The main engineering objective is to create a more compact and dynamic structure rather than simply replacing LEGO parts with printed parts. The custom geometry provides greater freedom to position components and optimise the robot's overall manoeuvrability.

3.3.5.4 Camera Field of View and Detection Rang

The Generation 3 camera configuration provided a wider field of view, but testing showed that the camera still could not detect objects at the required distance as effectively as intended. A wider field of view improved the visible area, but increasing width alone did not sufficiently improve the detection range. For Generation 4, a custom 3D-printed camera holder will be developed. The holder maintains the required wide viewing angle while positioning the camera at a higher position. This increases the camera's viewing distance while retaining the wider field of view required for obstacle detection. The design therefore represents a trade-off between field of view and detection distance rather than simply increasing one parameter.

3.3.5.5 Cable Management and Internal Component Layout

Testing and assembly also showed that the placement of the multiplexer and sensor cables could affect the cleanliness and organisation of the robot. Loose or poorly routed cables may interfere with moving components, particularly around the steering and driving mechanisms .For Generation 4, the central area of the chassis will be used as a dedicated cable-management region. The multiplexer and related wiring will be routed through the centre of the robot instead of being distributed around the moving mechanisms. This creates a cleaner internal layout and reduces the possibility of cables interfering with robot movement. The redesign also improves accessibility and reproducibility because the position of major electronic components and cable routes can be defined as part of the chassis design.

3.3.5.6 Engineering Decision Summary and Implications for Generation 4

Generation 3 testing showed that the steering wheel had limited grip, which could cause slight wheel slip during steering. For Generation 4, a higher-grip wheel with the same size will be used. This improves traction while keeping the existing steering geometry unchanged. The trade-off is slightly higher rolling resistance, but improved steering consistency is prioritised. The gyro also showed heading errors during testing. To improve the heading reference, the gyro will be moved to the front-centre of the robot, above the steering mechanism. This provides a more consistent reference for heading measurement, although it requires changes to the chassis and component layout. The LEGO chassis limited the freedom to optimize the robot's structure and component placement. Therefore, Generation 4 will use an integrated 3D-printed design for most of the robot. This provides greater freedom to create a more compact and dynamic structure, at the cost of reduced modularity and higher manufacturing effort. Although the

Generation 3 camera provided a wider field of view, its detection distance was still limited. A custom 3D-printed camera holder will raise the camera while maintaining its wide view. This improves viewing distance but requires recalibration because the camera perspective changes. Finally, the wiring will be routed through the centre of the robot, with the multiplexer and related cables organised in this area. This reduces the risk of cables interfering with moving parts and creates a cleaner system layout, although the central area becomes more compact and less accessible.

4.0 Generation 4: Design Optimisation and Control System Development

4.1 Overview

Generation 4 focused primarily on improving the robot's mobility and mechanical performance by addressing the limitations identified in Generation 3. Key improvements involved chassis design, steering performance, wheel traction and sensor placement optimisation. The existing software architecture was largely maintained, with necessary adjustments made to accommodate the updated mechanical and sensor configurations. These improvements aimed to enhance manoeuvrability, sensor reliability and overall autonomous navigation performance.

Generation 3 Limitations and Generation 4 Improvements

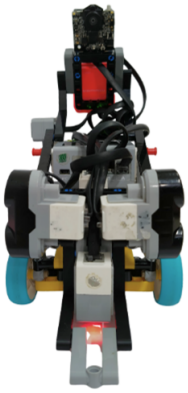
No.	Generation 3 Limitations	Generation 4 Improvements
1	Insufficient steering wheel traction, affecting directional stability.	Introduced higher-grip steering wheels to improve traction and steering stability.
2	Gyroscope heading errors affected directional accuracy.	Relocated the gyroscope to the front-centre position above the steering mechanism.
3	LEGO chassis limitations restricted mechanical design flexibility.	Developed an integrated 3D-printed chassis for greater design flexibility and component integration.
4	Camera detection range was insufficient despite its wide field of view.	Raised the camera mounting position to improve detection range while maintaining a suitable field of view.
5	Internal cable arrangement caused potential interference with mechanical components.	Improved cable routing and centralised the multiplexer to achieve a more organised internal layout.

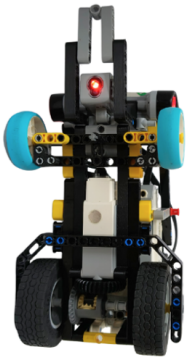
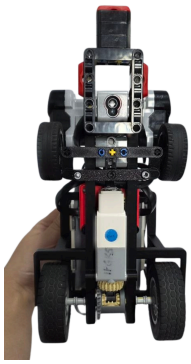
4.2 Mechanical Redesign and Mobility Enhancement

Generation 4 introduced key mechanical improvements to address the limitations identified in Generation 3, focusing on a more compact and integrated chassis, improved steering traction and enhanced manoeuvrability. A custom 3D-printed chassis was developed to optimize component arrangement, sensor positioning and cable management, while smaller rubber front tyres were adopted to improve grip and flexibility during the Obstacle Challenge. These modifications resulted in a lighter, more rigid and organised structure, providing a stronger foundation for sensor integration and autonomous navigation.

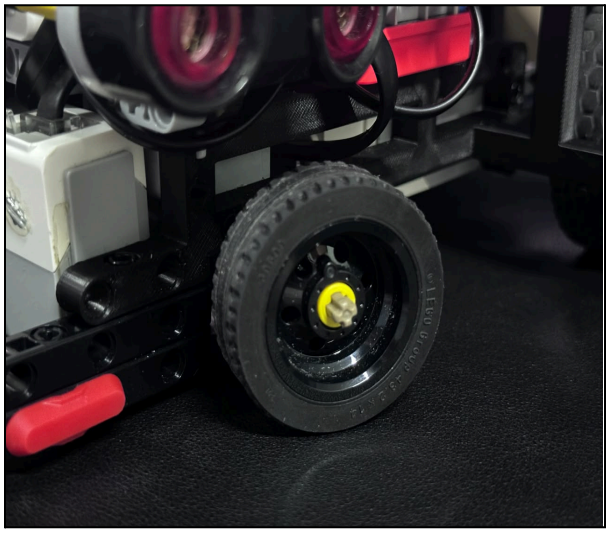
4.2.1 Mechanical Design Comparison Between Generation 3 and Generation 4

The mechanical configuration of Generation 4 was redesigned to overcome the structural and mobility limitations observed in Generation 3. The previous LEGO-based chassis restricted the flexibility of component positioning and internal cable arrangement. In Generation 4, a customised 3D-printed chassis was introduced to provide a more integrated structural platform. The front steering wheels were also replaced with smaller rubber tyres to improve traction and manoeuvrability. In addition, the new chassis allowed greater flexibility in positioning the gyroscope and camera according to their functional requirements. These changes represented a transition from a primarily LEGO-based construction towards a customised mechanical design with improved structural integration.

Gen 3		Gen 4
 Front View		Front View
Back View		Back View

Left View	→	Left View
Right View	→	Right View
Top View	→	Top View
 Bottom View	→	 Bottom View

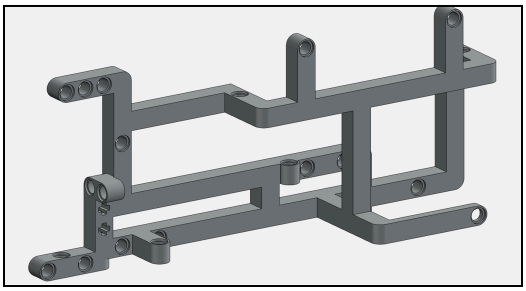
4.2.2 Wheel Selection and Traction Improvement



For Generation 4, a pair of front tyres from the LEGO Technic 42173 Koenigsegg Jesko Absolut Grey Hypercar was selected to improve steering traction and manoeuvrability. The selected narrow tyre (Part 30699) has a nominal diameter of 43.2 mm and a width of 14 mm.Compared with the previous wheel configuration, the selected tyres provide a rubber contact surface with a tread pattern, offering better grip against the competition surface. This helps reduce wheel slippage during steering and directional changes, allowing the robot to maintain more consistent movement.The relatively compact dimensions of the tyres also provide a suitable configuration for the front steering assembly. Their smaller profile supports flexible steering movements and manoeuvrability, particularly when navigating around obstacles during the Obstacle Challenge.

4.2.3 Integrated 3D-Printed Chassis Design

Generation 4 introduced a custom-designed, integrated 3D-printed chassis to overcome the structural and layout limitations of the previous LEGO-based construction. The chassis consists of two printed side sections connected through dedicated mounting interfaces, allowing components such as the wheel hubs, sensors and motors to be secured using LEGO Technic connectors.

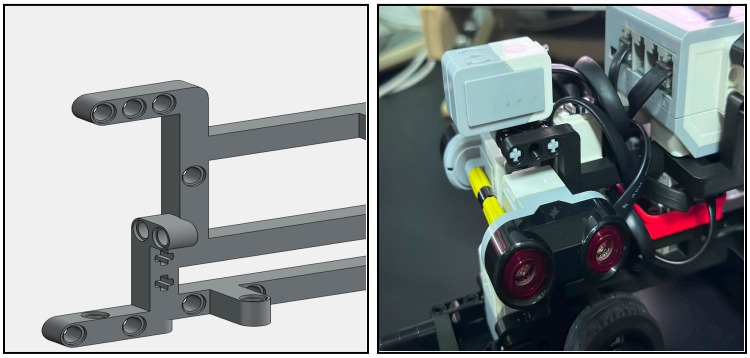


This design adopts a modular plug-and-play assembly approach, enabling individual components to be installed, removed and repositioned without requiring major modifications to the entire chassis. The use of integrated mounting features also provides greater flexibility in designing the shape and dimensions of the robot according to specific component requirements. Compared with the previous construction, the integrated chassis provides a more rigid and organised structural arrangement while reducing unnecessary structural complexity. Its customised geometry also creates additional space for internal cable routing, helping to minimise cable interference with moving mechanical components. Furthermore, the 3D-printed structure allows the chassis weight and component mounting positions to be customised according to the robot's mechanical requirements. Overall, this redesign provides a more compact, lightweight and adaptable platform for integrating the robot's mechanical, electrical and sensor systems.

4.2.4 Sensor Relocation and Custom Camera Mount Design

The customised 3D-printed chassis provided greater flexibility in sensor placement and mounting design. In Generation 4, the gyroscope was relocated to the front-centre section of the robot, directly above the steering mechanism. This position was selected to provide a stable mounting arrangement and a consistent reference for monitoring the robot's heading during movement. In addition, a customised camera mounting bracket was designed and 3D-printed to support the camera at the upper rear section of the robot. Unlike the previous mounting arrangement, the new bracket allowed the camera's height and viewing angle to be adjusted according to the robot's detection requirements.

4.2.4.1 Gyroscope Relocation and Mounting Design



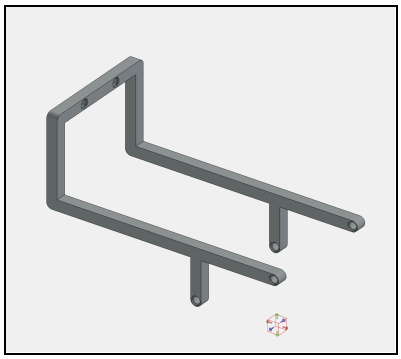
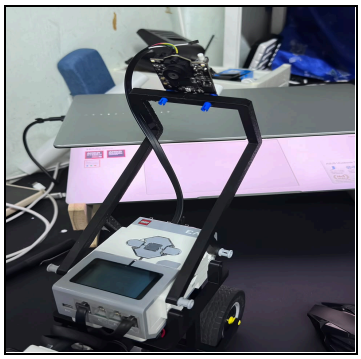
In Generation 3, the gyroscope was mounted at the rear of the robot due to limited mounting space, resulting in unstable readings and unexpected fluctuations during operation. In Generation 4, the gyroscope was relocated to the front-centre position above the steering mechanism and secured to a customised 3D-printed mount using a LEGO Technic axle. This provided a more stable mounting arrangement and improved the consistency of gyroscope readings during navigation.

Gyroscope Reading Stability Comparison Between Generation 3 and Generation 4

Round	Gen 3 Initial Reading (°)	Gen 3 Post-Run Reading (°)	Gen 4 Initial Reading (°)	Gen 4 Post-Run Reading (°)
1	0	4	0	0
2	0	7	0	0
3	0	9	0	0
4	0	9	0	0
5	0	14	0	0

Testing Description: The gyroscope reading was recorded 20 seconds after the robot completed one full mission and returned to the Parking Lot. The recorded value represents the gyroscope's accumulated angular reading after each completed mission. Five consecutive rounds were conducted to compare the reading consistency between Generation 3 and Generation 4.

4.2.4.2 Custom 3D-Printed Camera Mount Design

In Generation 3, the camera position was restricted by the LEGO chassis, limiting the flexibility to adjust its height and viewing angle. In Generation 4, a customised 3D-printed camera holder was designed to mount the camera at the upper rear section of the robot. This allowed different mounting heights and angles to be tested to find the most suitable position for object detection. The final configuration was selected based on its field of view and detection performance, providing greater flexibility in camera positioning for the Obstacle Challenge.

Camera Detection Range at Different Pillar Positions

Test	Forward Distance (a) (cm)	Lateral Distance (b) (cm)	Actual Diagonal Distance (c) (cm)	Detection Result
1	5	5	7.1	**Successful**
2	10	10	14.1	**Successful**
3	15	15	21.2	**Successful**
4	20	20	28.3	**Successful**
5	25	25	35.4	**Successful**
6	30	30	42.4	**Successful**
7	35	35	49.5	**Successful**

The diagonal detection test was conducted to evaluate the camera's ability to detect pillars positioned away from the robot's centreline. The pillar was placed at different forward and lateral distances, with the actual diagonal distance calculated using the Pythagorean Theorem. This

testing arrangement simulated pillars appearing at different diagonal positions within the camera's field of view. The results were used to assess the camera's detection capability when the robot approached pillars from different directions. Based on both tests, the camera successfully detected pillars at all tested positions, covering both straight and diagonal arrangements. The results demonstrated that the customised camera holder provided a suitable mounting position and viewing angle for pillar detection. This configuration was therefore selected for Generation 4 to support object detection during autonomous navigation.

4.3 Software Architecture

Generation 4 retained the existing programming architecture developed in Generation 3. The established program was reused to control the robot's movement, steering mechanism, sensor operations and autonomous navigation. Since the Generation 3 programming had already demonstrated satisfactory performance, no major changes to the software architecture were required. The development of Generation 4 focused mainly on improving the robot's mechanical design, mobility and sensor positioning. The existing program continued to operate with the updated hardware configuration, allowing the team to concentrate on mechanical improvements without rebuilding the entire control system.

Software Comparison of Generation 3 and Generation 4

Software Component	Generation 3	Generation 4
Robot Movement Control	Implemented	Retained
Steering Control	Implemented	Retained
Sensor Reading	Implemented	Retained
Obstacle Detection	Implemented	Retained
Autonomous Navigation	Implemented	Retained

4.4 Open Challenge Performance Evaluation

Trial	Direction	Zone	Completion Status	Completion Time
1	**Clockwise**	1	Successful	23s
2	**Counterclockwise**	2	Successful	25s
3	**Clockwise**	3	Successful	29s
4	**Counterclockwise**	4	Successful	22s
5	**Clockwise**	5	Successful	24s
6	**Counterclockwise**	6	Successful	31s

Generation 4 was evaluated through six trials in the Open Challenge, covering all six zones and both clockwise and counterclockwise driving directions. The robot successfully completed all six trials, achieving a 100% success rate. The completion times ranged from 22 to 31 seconds, with an average completion time of 25.7 seconds. The results demonstrate that the robot was able to navigate through all six zones while maintaining successful operation in both driving directions. The variation in completion time may be influenced by differences in driving direction, zone configuration and navigation conditions. Overall, the results indicate that Generation 4 achieved consistent task completion during the Open Challenge evaluation.

4.5 Obstacle Challenge Performance Evaluation

Trial	Direction	Amount of Obstacle	Parking Zone	Completion Status	Completion Time
1	**Clockwise**	7	1	Successful	90s
2	**Counterclockwise**	6	2	Successful	86s
3	**Clockwise**	7	3	Successful	92s
4	**Counterclockwise**	7	4	Successful	95s

Generation 4 was further evaluated through four trials in the Obstacle Challenge, involving both clockwise and counterclockwise driving directions. The robot successfully completed all four

trials, achieving a 100% success rate. During testing, the robot encountered between six and seven obstacles and successfully reached the designated parking zones in all four trials. The completion times ranged from 86 to 95 seconds, with an average completion time of 90.8 seconds. These results demonstrate the robot's ability to detect and navigate around multiple obstacles while following different driving directions. Furthermore, successful parking in all four designated zones indicates that the robot was able to complete the obstacle navigation task and reach its intended parking destination. Overall, the results show that Generation 4 successfully completed all tested Obstacle Challenge configurations, demonstrating its obstacle navigation and parking capabilities.

4.6 Final Design Overview

Generation 4 was developed to address the mechanical, mobility and sensor integration limitations identified in Generation 3. The primary objective was to produce a more compact, lightweight and mechanically organised robot while improving its manoeuvrability and sensor positioning for both Open Challenge and Obstacle Challenge.

4.6.1 Mechanical Design and Mobility

The most significant mechanical improvement was the introduction of a customised integrated 3D-printed chassis. The structure consists of two printed side sections connected using LEGO Technic components, providing dedicated mounting positions for the motors, hubs and sensors. This modular arrangement improved component organisation, structural integration and cable management. In addition, smaller rubber steering tyres from the LEGO Technic 42173 set were introduced to improve traction and provide greater steering flexibility during navigation.

4.6.2 Sensor Architecture and Integration

Sensor positioning was further refined to improve the robot's sensing configuration. The gyroscope was relocated from the rear to the front-centre position above the steering mechanism and secured using a customised 3D-printed mount and LEGO Technic axle. Five consecutive rounds of testing showed stable post-run readings under the recorded test conditions. Meanwhile, a customised adjustable camera holder was installed at the upper rear section of the robot, allowing different mounting heights and viewing angles to be evaluated. Camera detection tests were conducted at both straight-ahead and diagonal pillar positions, covering distances from approximately 5 cm to 50 cm.

4.6.3 Power System and Software Architecture

The power budget analysis evaluated the voltage, current and power requirements of the robot's electrical components. The robot uses six rechargeable 1.5 V lithium AA batteries, providing a nominal 9 V supply and a stated total energy capacity of 21 Wh. The software architecture was largely retained from Generation 3, with necessary adjustments made to accommodate the updated mechanical structure and sensor configurations. This allowed the team to focus primarily on mechanical optimisation without introducing major changes to the existing control system.

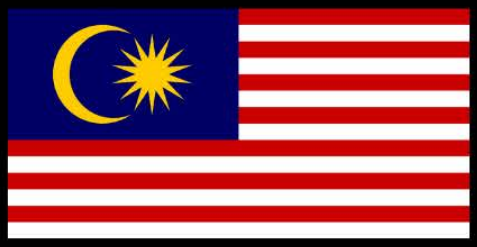
4.6.4 Performance Evaluation

Generation 4 was evaluated in both competition categories. In the Open Challenge, the robot completed all six trials across six zones, covering clockwise and counterclockwise directions, with a 100% success rate and an average completion time of 25.7 seconds. In the Obstacle Challenge, the robot successfully completed all four trials, navigating between six and seven obstacles and reaching all four designated parking zones. It achieved a 100% success rate with an average completion time of 90.8 seconds.

4.6.5 Overall Evaluation

Generation 4 brings together the mechanical redesign, sensor repositioning, power analysis and software integration developed throughout this iteration. The successful completion of all conducted Open Challenge and Obstacle Challenge trials demonstrates that the robot was capable of performing the tested tasks under the evaluated conditions. The final design provides a more integrated mechanical platform and reflects the improvements made in response to the limitations identified in previous generations.

For detailed information about our final robot, please refer to our WRO Future Engineers 2026 Engineering Journal.



WRO
WORLD ROBOT OLYMPIAD™

ROBOTS MEET CULTURE

FUTURE ENGINEERS

A SELF-DRIVING CAR CHALLENGE

INSTITUT PENDIDIKAN GURU
KAMPUS SARAWAK

Institute Of Teacher Education，
Sarawak Campus
Jalam Bakam，98009 Miri，
Sarawak，Malaysia
ipgksarawak@ipgm.edu.my